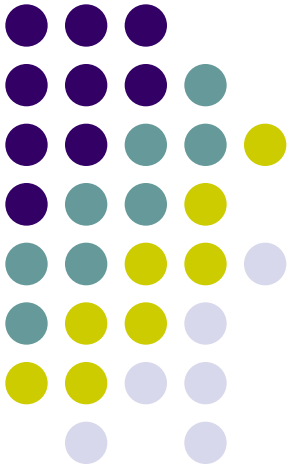
Natural Language Processing

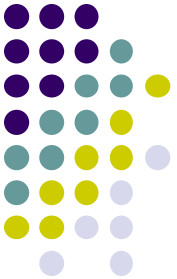
CPSC 488

Sequence Modeling

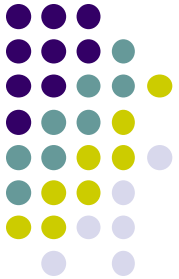
using **Recurrent Neural Networks**



Lecture Overview



- **Sequence modeling**
  - Learning sequential data and applications
  - From word meaning to sentence meaning
  - Encoding and decoding process
- **Neural models for sequence modeling**
  - Recurrent Neural Network (RNN)
  - Long Shot-Term Memory (LSTM)
  - Gated Recurrent Unit (GRU)
- **Issues of deep networks**
  - Vanishing gradients and exploding gradients



Sequential Data and Sequence Modeling

Sequential Data

- **What are sequential data?**
  - Any data that contain elements ordered into *sequences* where the elements depend on each other.
    - Text data, time-series data, speech or audio samples, songs, actions in a video stream, DNA sequence, etc.



even stage, and preserves its banks without washing. Its outlet by an easy channel enters the Sound almost within the present city limits.

Bordered with fertile lands, backed with inexhaustible forests of finest timber and immense fields of coal, iron and the precious minerals, the situation of this lake has not failed to attract the attention of every one versed in the ways of business or commerce who has visited the region.

Another such body of fresh water, lying within two miles of a noble ocean roadstead with which it may easily be joined, capable of being converted into anchorage ground for the fleets of the world, does not exist elsewhere; it would seem to have been provided by nature as a complement to our harbor system, which it will make perfect in its every feature.

A Focus of Great Enterprises.

Puget Sound having been for many years marked as the seat of coming great commercial expansion has become, within the last decade, the focus of enterprise looking to the realization of its future. Four great transcontinental railway lines have selected its waters for termini of their systems. Already two of them are establishing ocean steamship lines to Asia and the Oriental islands.

From all parts of the world come fleets of vessels to this island sea to compete for the cargoes which lie close at hand or at-

The coal measures are practically limitless in extent and the business of shipping coal is in its infancy. The amount of coal mined hereafter will be limited only by the demand for it.

Iron ores of the best quality—fit for steel making—cover an immense field. Works for its manufacture are in course of construction at several points.

The precious metals are distributed over a wide area and promise enormous development of wealth. Gold, silver, lead, copper, tin, asbestos, graphite, limestone, marble, granite, sandstone, are to be added to iron and coal. All these combined furnish an opulence of mineral resources rarely encountered in a single field.

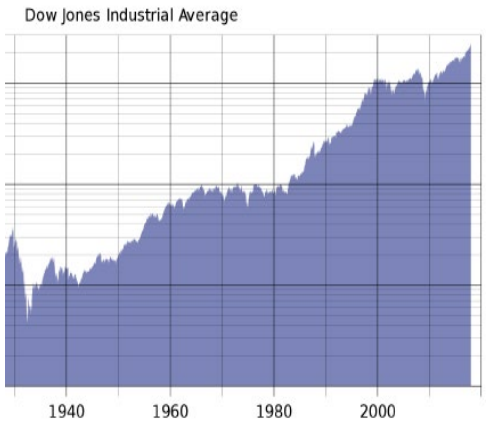
A state possessing unrivaled agricultural advantages, and having superadded to these such incomparable wealth in timber and minerals, cannot fail to develop manufacturing interests on a gigantic scale. Such development is already entered upon.

With the completion of the railway systems now speeding to their terminals at this point, the state will have an equipment of transportation lines greater and more perfect than any one state has hitherto equipped. She will be prepared to handle with utmost dispatch the traffic which throngs at her gates.

Scope of Foreign Commerce.

Great as the outlook for domestic trade may appear, it does not exceed the just

Text document



Time series data



A man riding a dirt bike on a dirt track

Some Issues with Sequential Data



even stage, and preserves its banks without washing. Its outlet by an easy channel enters the Sound almost within the present city limits.

Bordered with fertile lands, backed with inexhaustible forests of finest timber and immense fields of coal, iron and the precious minerals, the situation of this lake has not failed to attract the attention of every one versed in the ways of business or commerce who has visited the region.

Another such body of fresh water, lying within two miles of a noble ocean roadstead with which it may easily be joined, capable of being converted into anchorage ground for the fleets of the world, does not exist elsewhere; it would seem to have been provided by nature as a complement to our harbor system, which it will make perfect in its every feature.

A Focus of Great Enterprises.

Puget Sound having been for many years marked as the seat of coming great commercial expansion has become, within the last decade, the focus of enterprise looking to the realization of its future. Four great transcontinental railway lines have selected its waters for termini of their systems. Already two of them are establishing ocean steamship lines to Asia and the Oriental islands.

From all parts of the world come fleets of vessels to this island sea to compete for the cargoes which lie close at hand or at-

The coal measures are practically limitless in extent and the business of shipping coal is in its infancy. The amount of coal mined hereafter will be limited only by the demand for it.

Iron ores of the best quality—fit for steel making—cover an immense field. Works for its manufacture are in course of construction at several points.

The precious metals are distributed over a wide area and promise enormous development of wealth. Gold, silver, lead, copper, tin, asbestos, graphite, limestone, marble, granite, sandstone, are to be added to iron and coal. All these combined furnish an opulence of mineral resources rarely encountered in a single field.

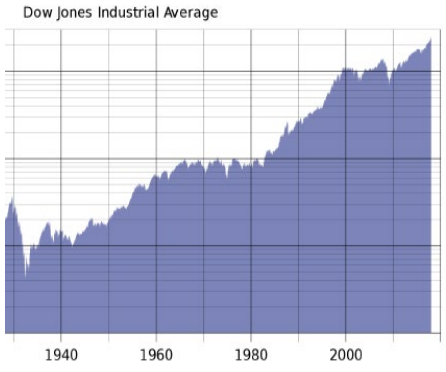
A state possessing unrivaled agricultural advantages, and having superadded to these such incomparable wealth in timber and minerals, cannot fail to develop manufacturing interests on a gigantic scale. Such development is already entered upon.

With the completion of the railway systems now speeding to their terminals at this point, the state will have an enjoyment of transportation lines greater and more perfect than any one state has hitherto equipped. She will be prepared to handle with utmost dispatch the traffic which throngs at her gates.

Scope of Foreign Commerce.

Great as the outlook for domestic trade may appear, it does not exceed the just

Text document



Time series data

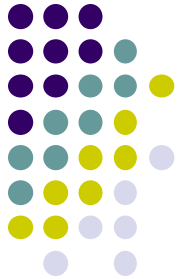
- **Dependencies between values in the input sequence**
  - e.g., a word can be strongly correlated with previous words in a sentence
  - May need to keep and utilize historical information
- **Varying length of input and output and temporal sequences**
  - A list of varying # of words within a sentence at each time step
- **Data without labels**
  - Unstructured text documents or a series of values
    - Need some ways to transform the input for training a model using supervised learning
    - We can use self-supervised learning.

Sequence Modeling



- **What is sequence modeling?**
  - To captures the underlying patterns and dependencies
- **What problems require sequence modeling? =>Applications**
  - Sequence classification
    - Assigning a label to the entire sequence
    - Text classification, speech recognition, activity recognition
  - Language modeling
    - Computing probabilities of character, word or sentence and predicting next character, word, phrase
  - Sequence-to-sequence (seq2seq) modeling
    - Machine translation, dialogue, question answering
  - **Sequence labeling**
    - Assigning a label to each element in a sequence
    - Part-of-Speech (POS) tagging, Named Entity Recognition (NER)
  - Natural language understanding
    - From **meaning of word** **to** **meaning of sentence**, paragraph, document, corpus
  - **Other problems**
    - Image/video captioning, Time-series data analysis, DNA sequence analysis, etc.

Methods to Learn Sequential Data



- **Autoregressive model**
  - Predicting the next term in a sequence from a fixed number of previous lagged observations (e.g., AR(1), AR(2)).
- **Linear dynamical system**
  - *Predicting the hidden states* using probability distribution (Kalman filter, successfully solving engineering problems)
- **Hidden Markov Model (HMM)**
  - *Inferring hidden state distribution* from transitions between hidden states and outputs using a transition matrix
- **Neural models**
  - Feed-forward neural networks as generalized autoregressive models without memory
  - Special neural network architectures: RNN, LSTM, Transformer
    - Neural networks with *special connections* to *store information about the past* to learn the relationships in data

Some Issues with Sequential Data



even stage, and preserves its banks without washing. Its outlet by an easy channel enters the Sound almost within the present city limits.

Bordered with fertile lands, backed with inexhaustible forests of finest timber and immense fields of coal, iron and the precious minerals, the situation of this lake has not failed to attract the attention of every one versed in the ways of business or commerce who has visited the region.

Another such body of fresh water, lying within two miles of a noble ocean roadstead with which it may easily be joined, capable of being converted into anchorage ground for the fleets of the world, does not exist elsewhere; it would seem to have been provided by nature as a complement to our harbor system, which it will make perfect in its every feature.

A Focus of Great Enterprises.

Puget Sound having been for many years marked as the seat of coming great commercial expansion has become, within the last decade, the focus of enterprise looking to the realization of its future. Four great transcontinental railway lines have selected its waters for termini of their systems. Already two of them are establishing ocean steamship lines to Asia and the Oriental islands.

From all parts of the world come fleets of vessels to this island sea to compete for the cargoes which lie close at hand or at-

The coal measures are practically limitless in extent and the business of shipping coal is in its infancy. The amount of coal mined hereafter will be limited only by the demand for it.

Iron ores of the best quality—fit for steel making—cover an immense field. Works for its manufacture are in course of construction at several points.

The precious metals are distributed over a wide area and promise enormous development of wealth. Gold, silver, lead, copper, tin, asbestos, graphite, limestone, marble, granite, sandstone, are to be added to iron and coal. All these combined furnish an opulence of mineral resources rarely encountered in a single field.

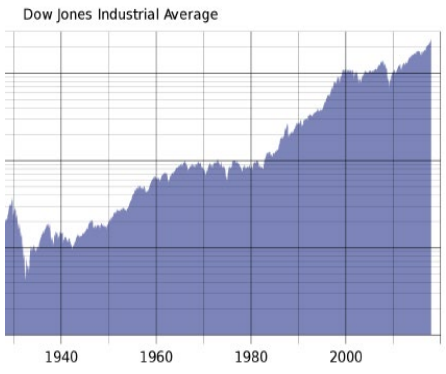
A state possessing unrivaled agricultural advantages, and having superadded to these such incomparable wealth in timber and minerals, cannot fail to develop manufacturing interests on a gigantic scale. Such development is already entered upon.

With the completion of the railway systems now speeding to their terminals at this point, the state will have an enjoyment of transportation lines greater and more perfect than any one state has hitherto equipped. She will be prepared to handle with utmost dispatch the traffic which throngs at her gates.

Scope of Foreign Commerce.

Great as the outlook for domestic trade may appear, it does not exceed the just

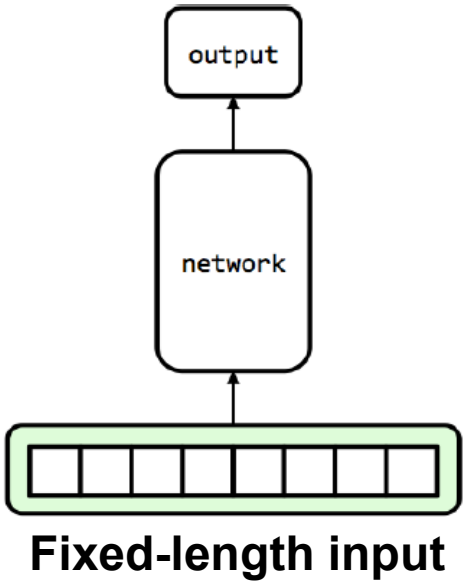
Text document



Time series data

- **Varying length of input and output and temporal sequences**
  - A list of varying # of words within a sentence at each time step
- **Dependencies between values in the input sequence**
  - e.g., a word can be strongly correlated with previous words in a sentence
  - May need to keep and utilize historical information
- **Data without labels**
  - Unstructured text documents or a series of values
    - Need some ways to transform the input for training a model using supervised learning
    - We can use self-supervised learning.

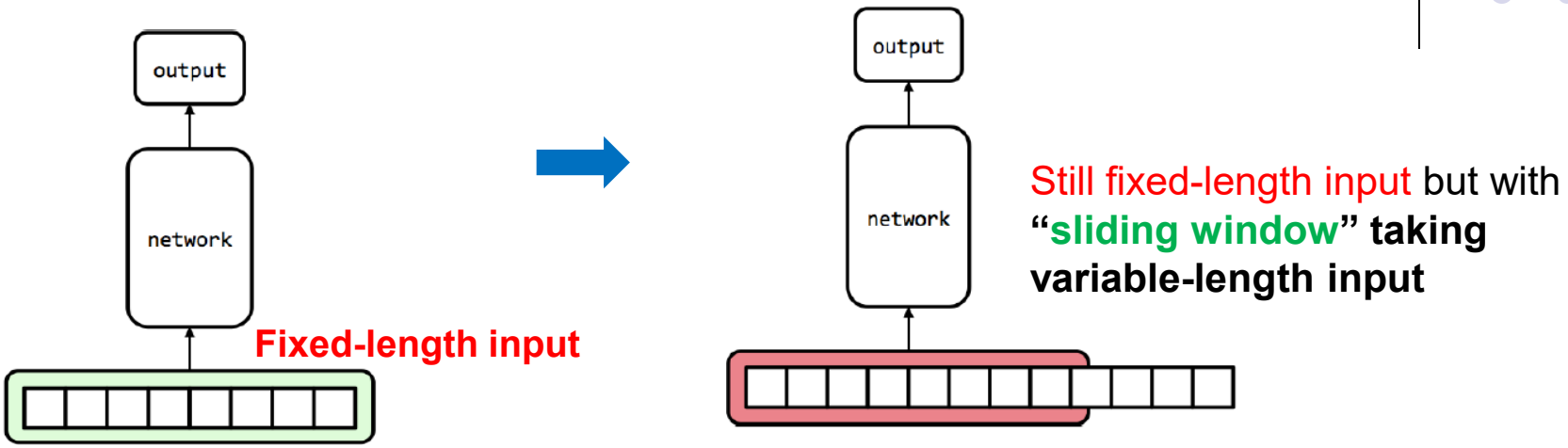
Fixed-length Input to Handle Variable-length Data



- **Feedforward network (FFN)** with **fixed-length input** (to **handle variable-length input**)
  - Truncating longer input sequences to fixed sequences
  - Padding the input with placeholder values (e.g., zeros) to make them all the same length

- **Issues of fixed-length input**
  - Difficult to take very long input sequence
  - Difficult to find the long-distance relationships

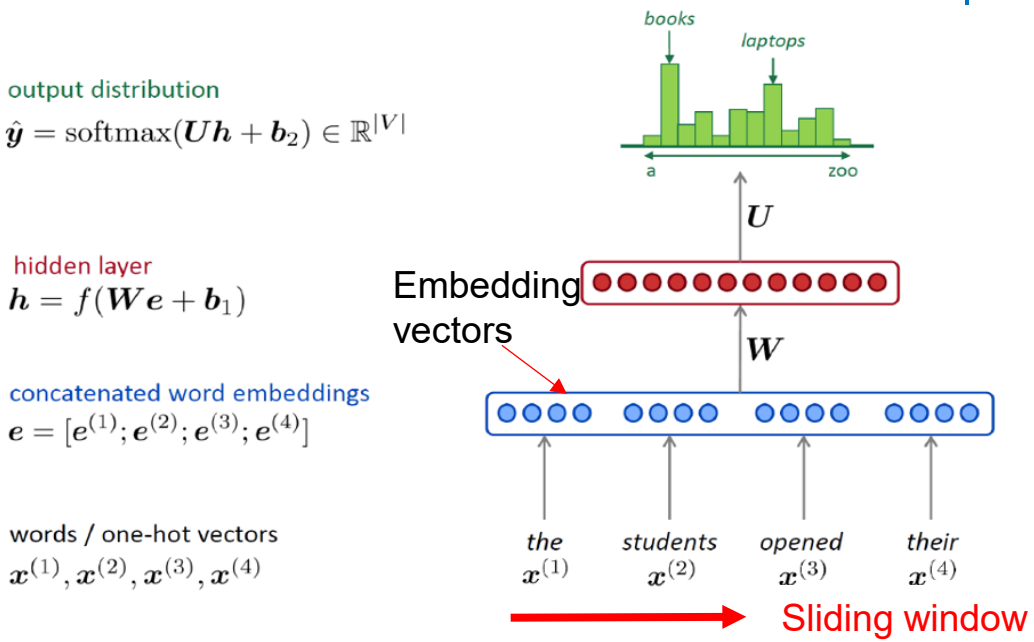
Fixed-length Input to Handle Variable-length Data



- **Other methods to handle variable-length input data**
  - Aggregating variable-length sequences into fixed-size feature vectors using mechanisms such as pooling or attention called "encoding"
  - Converting each input into a fixed-size vector called "embedding"
    - Can handle dependencies in long sequences
  - **Special neural network architectures for encoding or embedding**
    - Recurrent Neural Network (RNN), Long Short-Term Memory (LSTM), GRU
    - Transformer

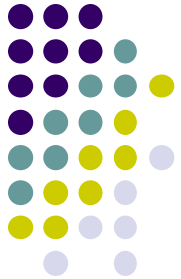
Feedforward Neural LM

(Bengio et al., 2003)



- **Issues of using sliding window**
  - A fixed number of previous words (no long-term dependencies, context truncation, poor generalization)
  - Computational inefficiency for long sequences (no memory or shared states between time steps)
  - Parameter explosion (due to concatenation of n embeddings, the input layer grows linearly with window size

- **A simple feedforward LM**
  - Input: a word sequence $\boldsymbol{w_1, w_2, \ldots, w_n}$
  - Output: a probability distribution of the next word $\boldsymbol{P(w_t | w_{1:t-1})}$
    - No sparsity better for contextual information (improvement over N-gram LM)
    - Motivated Word2vec embeddings



Recurrent Neural Networks (RNNs)

Simple Recurrent Neural Network (RNN)

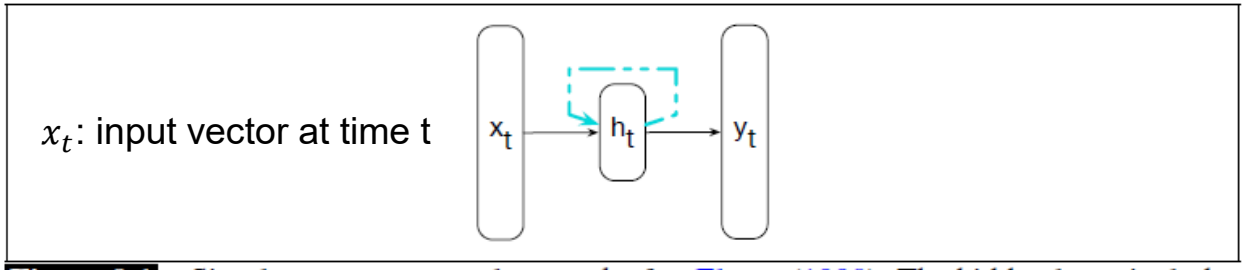


Figure 9.1 Simple recurrent neural network after Elman (1990). The hidden layer includes a recurrent connection as part of its input. That is, the activation value of the hidden layer depends on the current input as well as the activation value of the hidden layer from the previous time step.

- **The basic idea of RNN**
  - Sequences are processed by presenting one item x_t at a time t.
  - The hidden layer with a recurrent connection from the previous time step provides a form of memory
  - Previous states to capture context or dependency information.

Simple Recurrent Neural Network

Feedforward Network
Each layer receives input from the previous layer with no loops

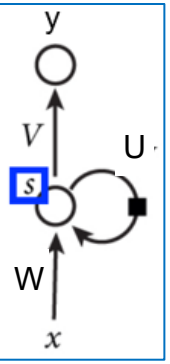
FFN

vs.



Recurrent Network
Each layer receives input from the previous layer and the output from the previous time step

RNN



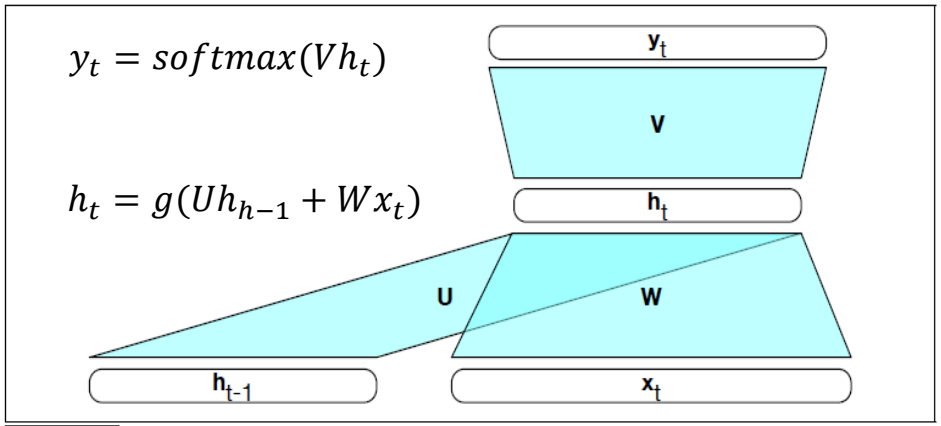
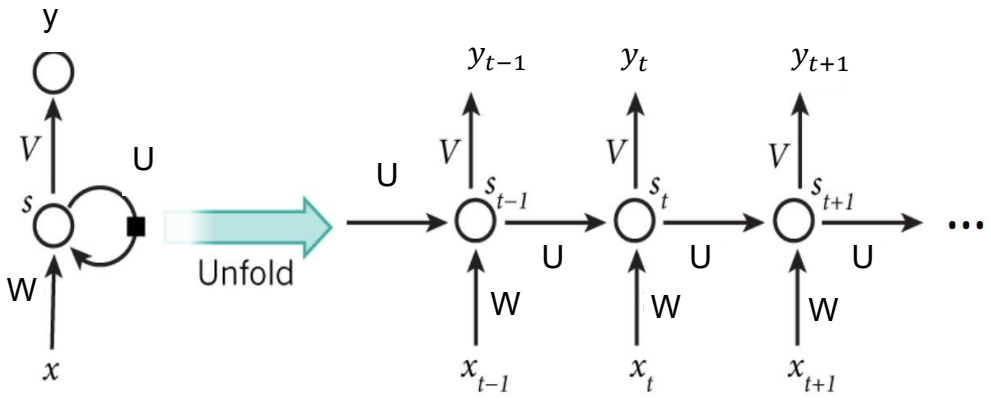


Figure 9.2 Simple recurrent neural network illustrated as a feedforward network.

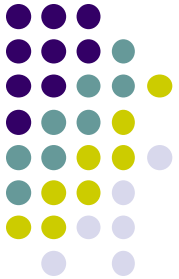
- **The differences between FFN and RNN**
  - The weight update for hidden layer: $h_t = g(Uh_{h-1} + Wx_t)$
    - The architecture becomes more complex than feedforward network.
  - RNN is a deep feedforward network (with many *conceptual* layers).
    - Other than this, everything else can be the same as the typical neural networks.

Simple RNN in Action (Unfolding)



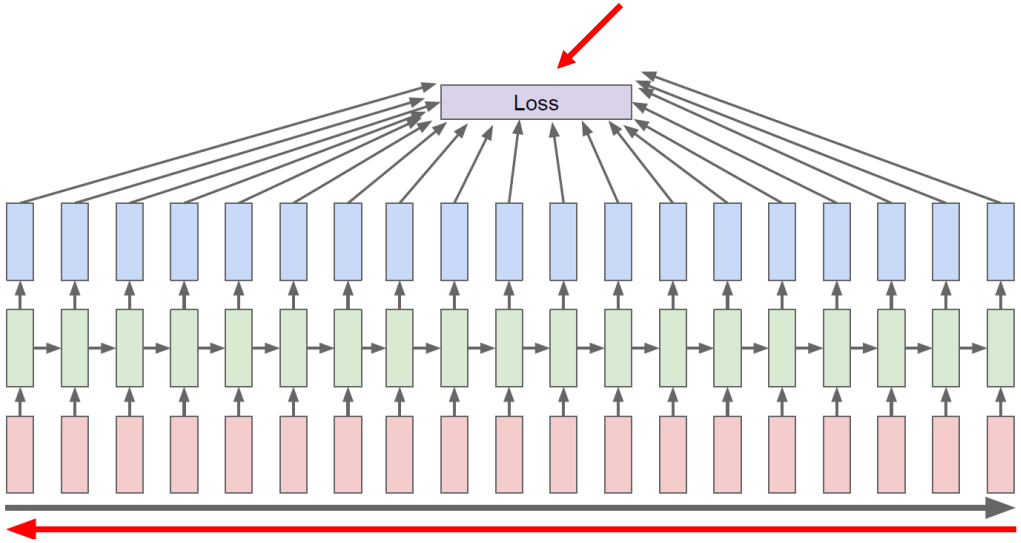
- **The sequential nature of RNN unrolling the network in time**
  - Network layers are recalculated for each time step.
  - But weight matrices **W**, **U**, and **V** are shared **across time**.
    - The same function, weights, and parameters are used at every time step for the entire sequence since the update occurs within the same neuron just at different time.
  - A RNN is a deep feedforward network with recurrent connections.
    - The model retains the **past information** via recurrent connections that depends on the input data and the weights rather than a fixed duration.

Training RNNs



- **Forward through the *entire sequence* to compute the *loss***
  - Compute h_t and y_t, accumulating the loss at each time step, saving the values of the hidden layer for the use at the next time step.
- **Backward through the *entire sequence* to update the *weights***
  - Backpropagate the errors of all the weights in reverse.
    - Since the weight across all the time steps is the same, we can combine gradients from all time steps.
  - **Do we need a specialized computational graph for RNN?**
  - **No**. We can perform "**Backpropagation** through **time**"
    - For an input sequence, generate an unrolled feedforward computational graph specific to that input, perform the ordinary backpropagation "through time", then apply any general-purpose gradient techniques to update the weights.
- **What can RNNs learn?**
  - Turing complete
    - Any function computable by a Turing machine can be computed by a RNN of a finite size (Siegelmann and Sontag, 1995).

Backpropagation through Time

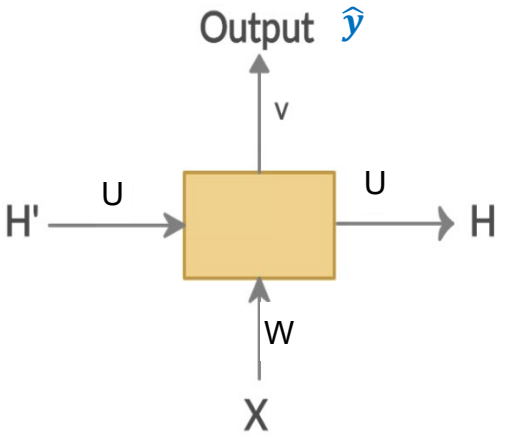


Forward through *the entire sequence* to compute *loss*, then

backward through the entire sequence to update *the weights with gradient*.

Weight Updates in RNNs

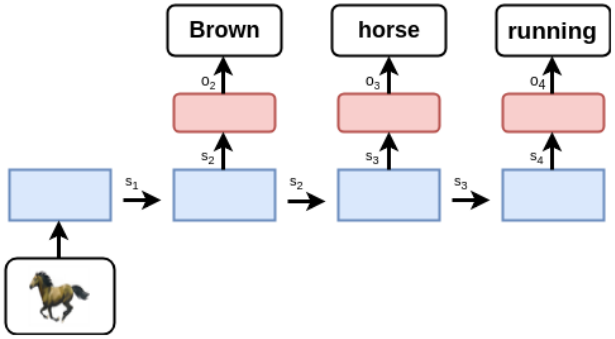
- **Forward propagation**
  - $H = wX + uH'$
  - $\hat{y} = vH$
- **Backpropagation through time**
  - Loss function $L(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$
  - $\frac{\partial L}{\partial v} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial v} = (\hat{y} - y)H = (\hat{y} - y)(wX + uH')$
  - $\frac{\partial L}{\partial u} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial H} \frac{\partial H}{\partial u} = (\hat{y} - y)vH'$
  - $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial H} \frac{\partial H}{\partial w} = (\hat{y} - y)vX$
- **Weight update rules**
  - $w_v^{t+1} = w_v^t - \alpha \frac{\partial L}{\partial v}$
  - $w_u^{t+1} = w_u^t - \alpha \frac{\partial L}{\partial u}$
  - $w_w^{t+1} = w_w^t - \alpha \frac{\partial L}{\partial w}$



RNN Architectures and Applications

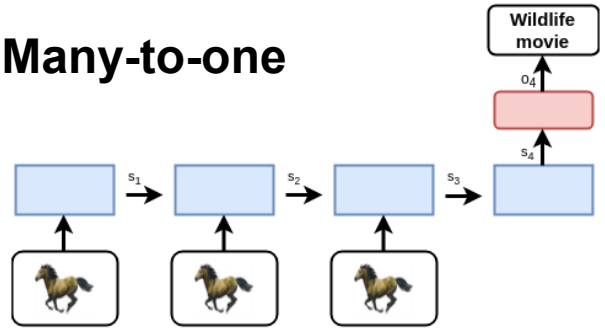
One-to-many

Image captioning



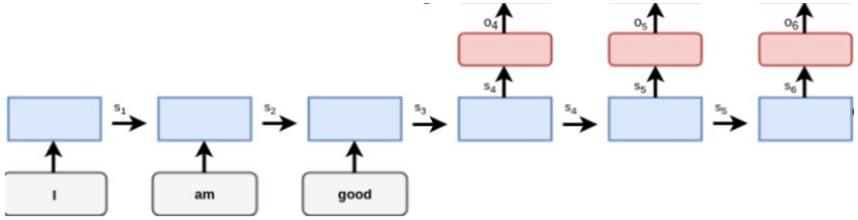
Many-to-one

Video classification or
sentiment classification



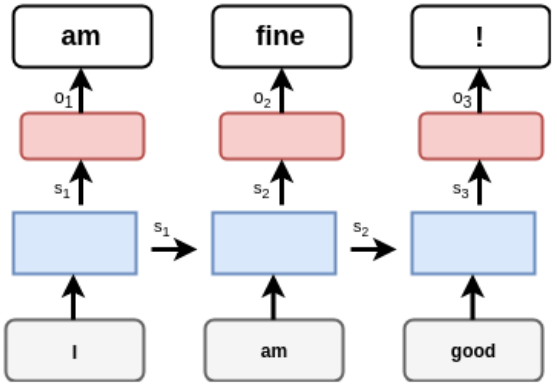
Many-to-many

Language translation



Many-to-many

Autocomplete system



More RNN Applications

- **Identifying successful spots in sports**
  - Successful spots or strategies in basketball, soccer, etc.
- **Image captioning**

Success cases



Two giraffes standing in a grassy field

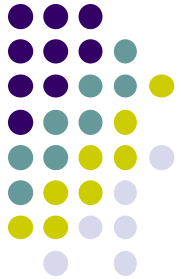


A man riding a dirt bike on a dirt track

Failure cases



A woman is holding a cat in her hand



RNNs for Sequence Classification

RNN Architectures for NLP Tasks

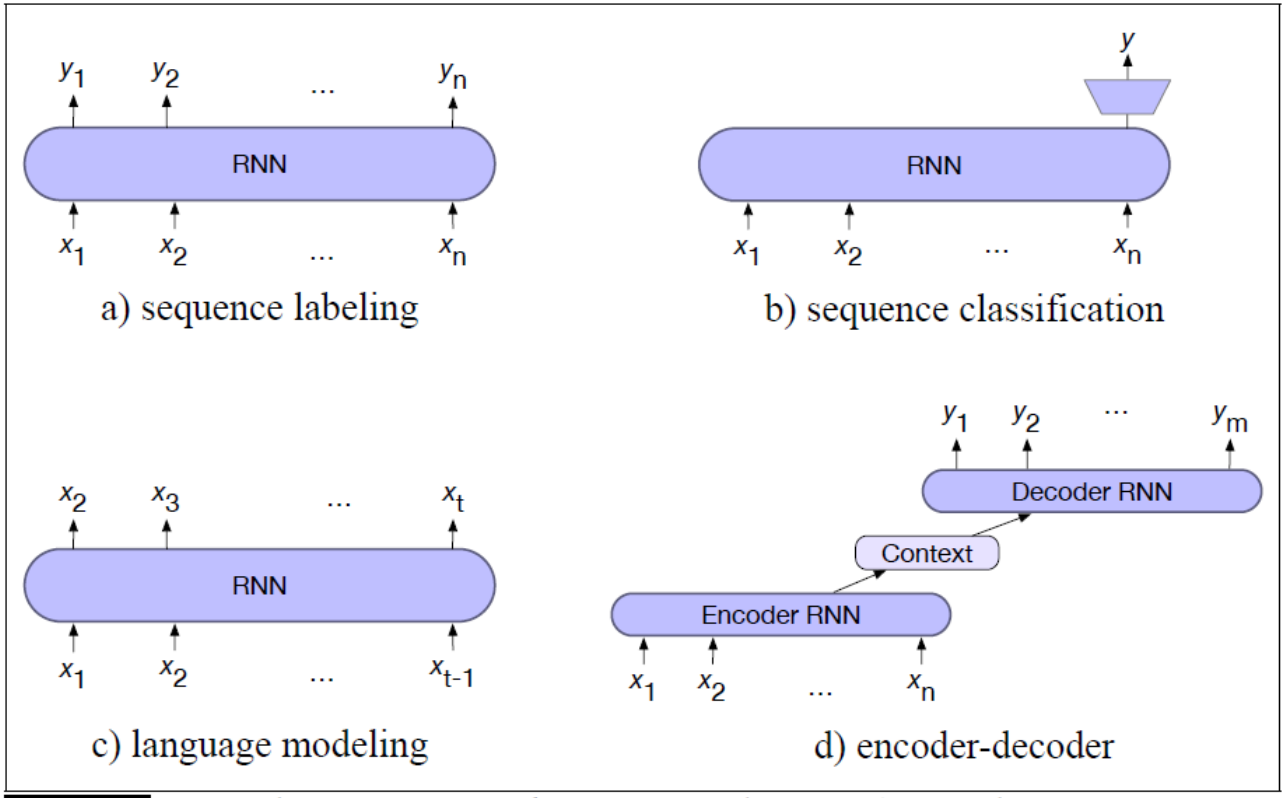


Figure 9.15 Four architectures for NLP tasks. In sequence labeling (POS or named entity tagging) we map each input token x_i to an output token y_i. In sequence classification we map the entire input sequence to a single class. In language modeling we output the next token conditioned on previous tokens. In the encoder model we have two separate RNN models, one of which maps from an input sequence **x** to an intermediate representation we call the **context**, and a second of which maps from the context to an output sequence **y**.

Sequence Classification

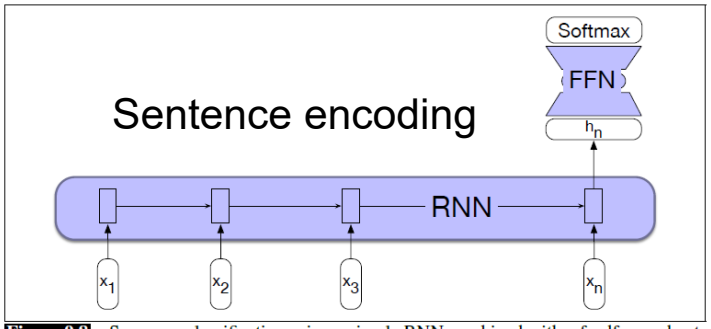


Figure 9.8 Sequence classification using a simple RNN combined with a feedforward network. The final hidden state from the RNN is used as the input to a feedforward network that performs the classification.

- **The task of assigning a label to an entire sequence**
  - For sentiment analysis, topic classification, intent classification, time-series analysis
- **Architecture**: A simple RNN + Feedforward Neural Network (FFN)
  - **Input**
    - Sentence, paragraph, document, image, prices
    - One word in a sentence at a time step t, proceed to the last term.
  - **Training**
    - **One** RNN node looping through at each time step
    - No intermediate outputs for each word
    - Pass the last hidden state h_n (encoding) to a feedforward network (FFN)
  - **Output**
    - Softmax output
    - The error is backpropagated to the weights of FFN and RNN.
    - Updating the weights only after the entire sequence (end-to-end training)

Sequence Encoding

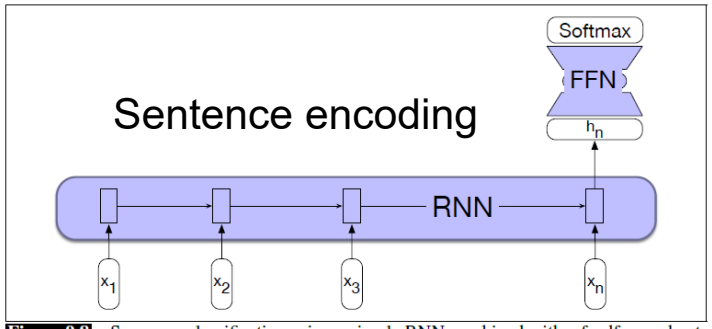
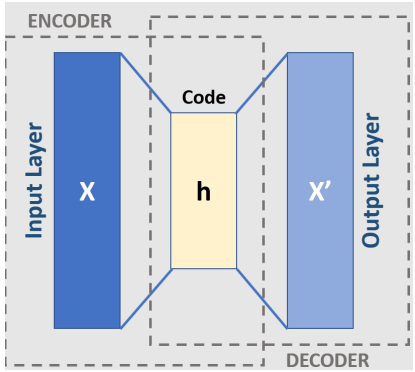


Figure 9.8 Sequence classification using a simple RNN combined with a feedforward network. The final hidden state from the RNN is used as the input to a feedforward network that performs the classification.

- **What is encoding?**
  - Converting an input (text) sequence to typically a compact code vector.
  - **Why encoding?**
    - To captures context, order, relationships, and generalize patterns
    - To compress information efficiently
- **Some options** for **sequence encoding** using hidden state $\boldsymbol{h_n}$
  - For a sequence of *n* tokens, RNN outputs $h_1, h_2, \ldots, h_n$ (*d*-dimension)
  - Option 1: h_n is a compressed representation of the entire sequence.
    - $h_i = f(x_i, h_{i-1})$
  - Option 2: Average all hidden states (mean pooling)
    - $h_n = h_{mean} = \frac{1}{n}\sum_i^n h_i$
  - Option 3: Take the element-wise max (max pooling)
    - $h_n = h_{max}[j] = \max(h_1[j], h_2[j], \ldots, h_n[j])$ for each dimension *j=1,2,…,d*

Autoencoder (Rumelhart, G. Hinton et al., 1987)



- **Network topology:** Symmetric Feedforward
  - **Encoder**: Input layer, hidden layer, bottleneck layer
    - Transforms the input into a compressed latent representation (code): $z = f(x)$
    - Enforce the network to learn the most essential features.
    - Representation learning
  - **Decoder**: Bottleneck layer, hidden layer, output layer
    - Reconstructs the original input from the latent representation: $\hat{x} = g(z)$
    - The decoder layers gradually increase dimensionality to match the input size.
- **Training**
  - Compute the reconstruction error between x and $\hat{x}$
    - Mean Squared Error (MSE), Mean Absolute Error, Binary cross-entropy, Hinge loss
  - Update the weights by backpropagation
    - To minimize the reconstruction error

Bidirectional RNNs (for sequence classification)

- The final hidden states of the **two independent** RNNs are combined from the forward and backward passes ($h_t = h_t^f \oplus h_t^b$).
- The combined output is the input to a subsequent FFN with Softmax for classification.

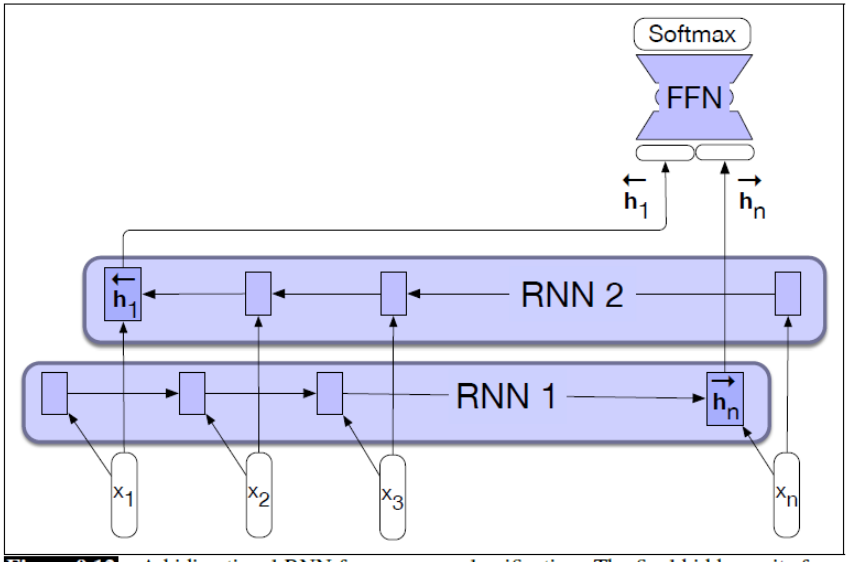
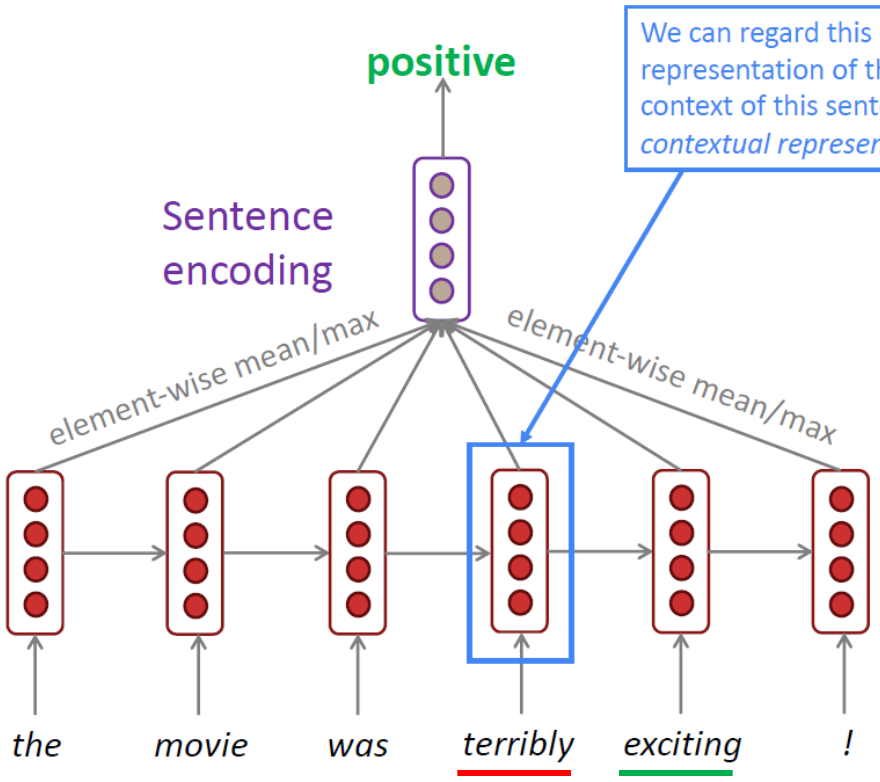


Figure 9.12 A bidirectional RNN for sequence classification. The final hidden units from the forward and backward passes are combined to represent the entire sequence. This combined representation serves as input to the subsequent classifier.

- **To learn the *context* from both directions**
  - When the entire input sequence is known, run two separate RNNs:
    - One left-to-right, another one right-to-left, and concatenate their representation
  - Bidirectional RNNs proven to be effective for sequence classification.

Example: Bidirectional RNNs for Sentiment Classification



We can regard this hidden state as a representation of the word *"terribly"* in the context of this sentence. We call this a *contextual representation.*

These contextual representations only contain information about the *left* context (e.g. *"the movie was"*).

What about *right* context?

In this example, *"exciting"* is in the right context and this modifies the meaning of *"terribly"* (from negative to positive)

Forward RNN: $h_t^f = RNN_{fw}(h_{t-1}, x_t)$

Backward RNN: $h_t^b = RNN_{bw}(h_{t+1}, x_t)$

Concatenated hidden states: $h_t = h_t^f \oplus h_t^b$

A Multilayer RNN: a Stack of RNNs

One RNN is already deep on **one dimension** over many timesteps.

We can make them "**deeper**" on another dimension by **combining** the feedforward nature of unrolled computational graph for complex architectures for more complex representations.

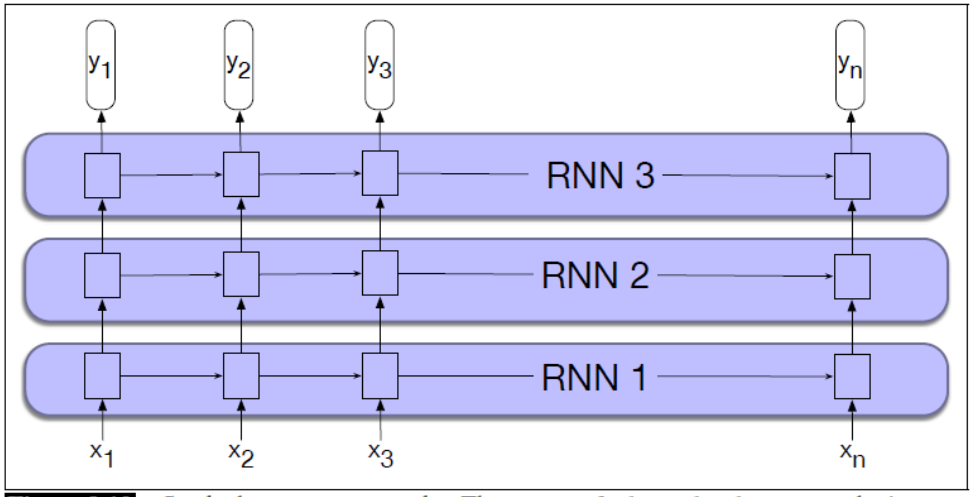
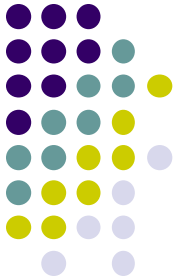


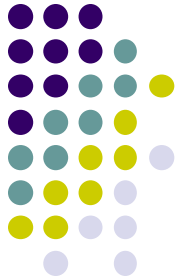
Figure 9.10 Stacked recurrent networks. The output of a lower level serves as the input to higher levels with the output of the last network serving as the final output.

- **Stacked RNNs**
  - Multiple RNNs generally outperforms a single-layer RNN.
    - The optimal number of layers is specific to each application, but the number of stacks increase the training cost.



RNNs for Language Modeling

Language Model (LM)



- **The language modeling problem**
  - Assign a probability to every sentence (or word, word sequences, or any strings)

    $P(W) = P(w_1, w_2, \dots, w_n)$ or $P(w_i | w_1, w_2, \dots, w_{i-1})$

  - Finite vocabularies (e.g., words, characters)
  - Infinite set of word sequences (e.g., sentences, phrases, paragraphs)
- **(Probabilistic) Language Model**
  - A **model** that **computes** the **probabilities** of words or sentences
  - Examples
    - **P**("disseminating so much currency") = 10^{-15}
    - **P**("spending so much currency") = 10^{-9}

RNN LM for Language Generation

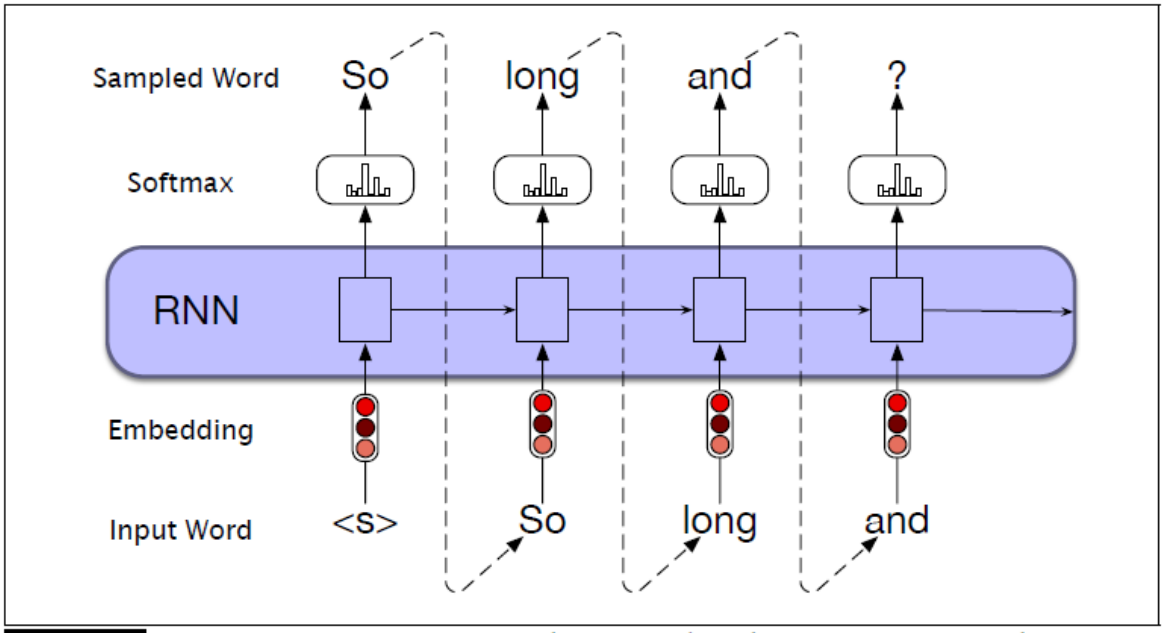


Figure 9.9 Autoregressive generation with an RNN-based neural language model.

Just like an **N-gram LM**, an **RNN** can generate text by repeated sampling.

Sampled output becomes next step's input.

- This simple architecture can generate a text.
  - The key is the generation component with an appropriate context, instead of simply predicting the next word, we can provide a richer task-oriented context.
- **Language generation** is one of the key capabilities for **generative AI**
  - Question answering, conversational dialogue, story generation, etc.
  - Text with other generation such as images, music, sounds, etc.

Training RNN for LM

Correct words (**ground-truth**)

Words are predicted one at a time.

Words are fed one at a time.

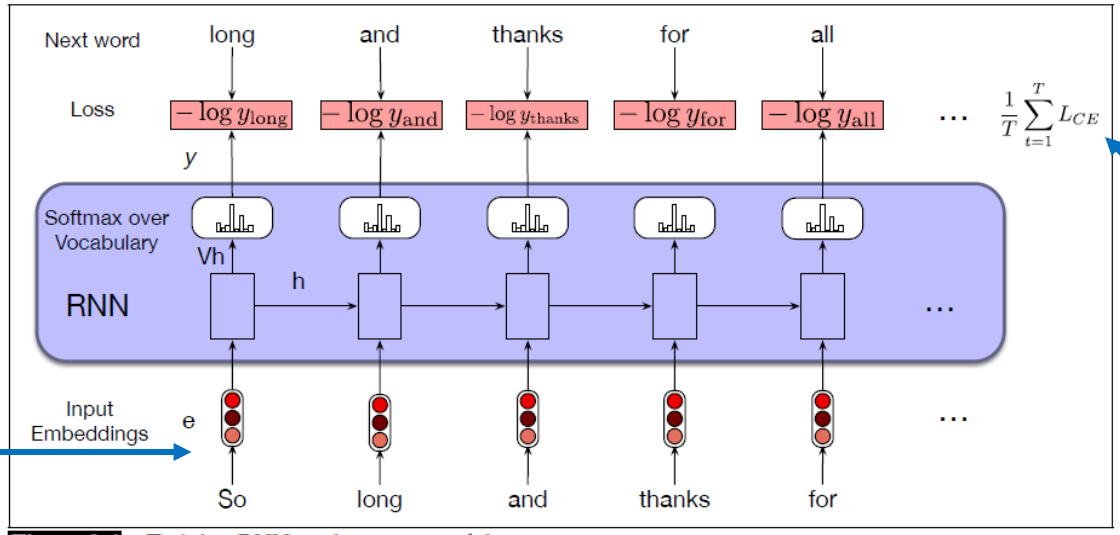
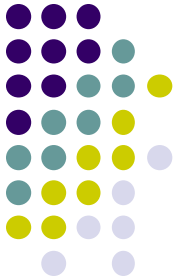


Figure 9.6 Training RNNs as language models.

Weights are updated all at once by the average error from **the entire sequence**.

- **Training an RNN for language model**
  - **Compute** the output probability distribution $\hat{y}_t$ for the next word w_{t+1}, and cross-entropy (CE) loss $L_{CE}(y_t, \hat{y}_t) = -log\hat{y}_t[w_{t+1}]$ with true next word instead of using the predicted word ("teacher forcing").
    - Note: $y_{t+1}[w_{t+1}]$ =1 in cross-entropy loss
    - Repeat this process for the entire sequence (or mini-batch).
  - **Compute** the average loss from the entire sequence: $\frac{1}{T}\sum_{t=1}^{T} L_{CE}$
  - **Adjust** the weights to minimize the average CE loss.
    - The weight rule: $\theta = \theta - \alpha \frac{\partial L_{CE}}{\partial \theta}$

Teacher Forcing



- **Analogy**
  - Multi-part exam grading with partial credits for learning purpose
- **Why teacher forcing?**
  - Typical RNN have recurrent connections => feedback input (output is fed back).
    - Problem: Input from incorrectly predicted output can lead to a compounding error over time.
  - Trick!
    - Provide the model with the correct word w_{t+1} at each time step t instead of $\hat{y}_{t-1}$.
    - The cross-entropy loss is computed based on the next word w_{t+1} (ground-truth) that will come at next time step.
- **Pros and cons of teacher forcing**
  - Pros
    - Stability of learning, faster convergence in sequence modeling
  - Cons
    - Exposure bias and discrepancy between training and inference
    - Scheduled sampling may address this issue.

Weight Tying in Training RNN LMs

Weight tying

$e_t = \boldsymbol{W} x_t$

$h_t = g(U h_{t-1} + W e_t)$

$y_t = softmax(\boldsymbol{W}^T h_t)$

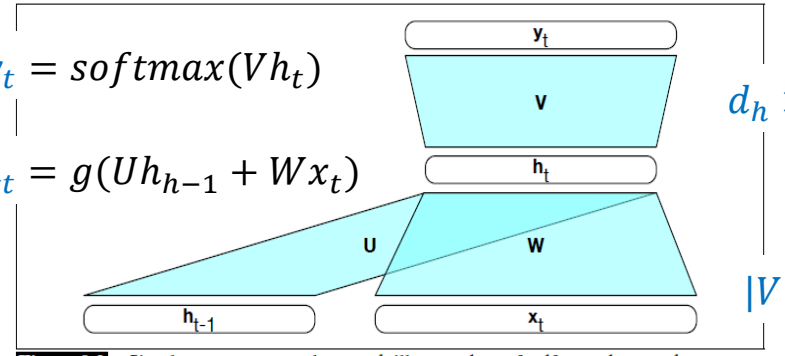


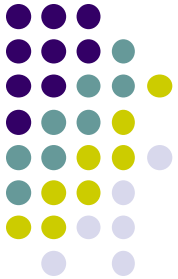
Figure 9.2 Simple recurrent neural network illustrated as a feedforward network.

$d_h \times |V|$ output layer matrix W'

$|V| \times d_h$ embedding matrix W

- **Embedding matrix** $W_{|V| \times d_h}$
  - W represent the word embeddings learned during the training process.
    - d_h is the dimension of hidden layer and $|V|$ is the size of vocabulary.
- **Output layer matrix** W'
  - W' is also, the learned word embeddings based on the input sequence.
  - W and W' will be similar. Is it necessary to have both?
- **Weight tying**
  - Simply uses a single set of embeddings W at the input and Softmax layers.
    - Sharing the same weights across different layers
    - To reduce number of parameters, regularization, improved model perplexity

Evaluation of RNN LMs



Compared to N-gram LMs

Model	Num. Params [billions]	Training Time [hours]	Training Time [CPUs]	Perplexity
Interpolated KN 5-gram, 1.1B n-grams (KN)	1.76	3	100	67.6
Katz 5-gram, 1.1B n-grams	1.74	2	100	79.9
Stupid Backoff 5-gram (SBO)	1.13	0.4	200	87.9
Interpolated KN 5-gram, 15M n-grams	0.03	3	100	243.2
Katz 5-gram, 15M n-grams	0.03	2	100	127.5
Binary MaxEnt 5-gram (n-gram features)	1.13	1	5000	115.4
Binary MaxEnt 5-gram (n-gram + skip-1 features)	1.8	1.25	5000	107.1
Hierarchical Softmax MaxEnt 4-gram (HME)	6	3	1	101.3
Recurrent NN-256 + MaxEnt 9-gram	20	60	24	58.3
Recurrent NN-512 + MaxEnt 9-gram	20	120	24	54.5
Recurrent NN-1024 + MaxEnt 9-gram	20	240	24	51.3

Table 1: Results on the 1B Word Benchmark test set with various types of language models.

Example RNN LM: Text Generation

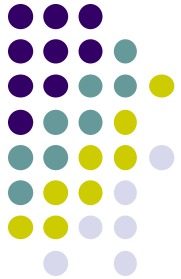


- You can train an RNN LM on any kind of text, then generate text in that style.
- RNN LM trained on Obama speeches:

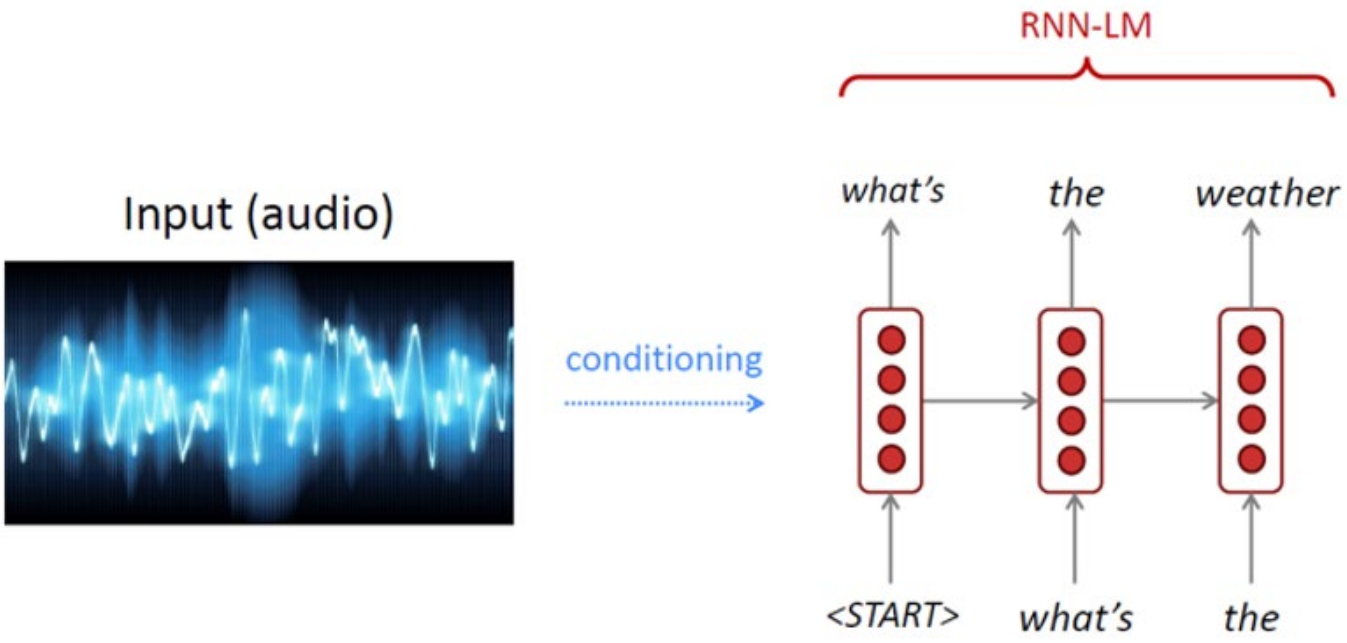


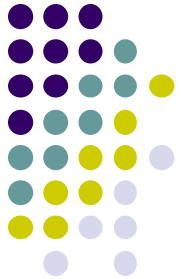
The United States will step up to the cost of a new challenges of the American people that will share the fact that we created the problem. They were attacked and so that they have to say that all the task of the final days of war that I will not be able to get this done.

Example RNN LM: Speech Recognition



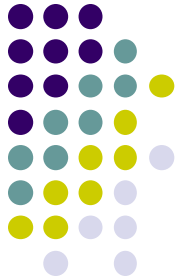
- **Speech recognition**





Sequence-to-Sequence Modeling

Sequence-to-Sequence Modeling



- **What is Sequence-to-Sequence (seq2seq) modeling?**
  - Mapping the input sequence to arbitrary length of contextually appropriate output sequence
- **Applications of seq2seq models**
  - Language modeling and text generation
    - To generate an output sequence, each token is predicted based on the previously generated tokens and the entire input sequence.
  - Machine translation
    - Example: “How are you?” => “Cómo estás”
  - Image captioning
  - Question answering
  - etc.

Main Components of seq2seq Models

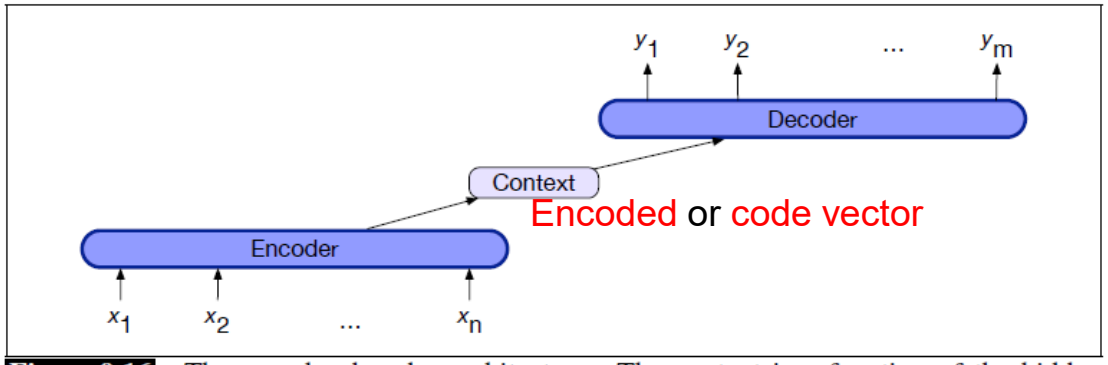
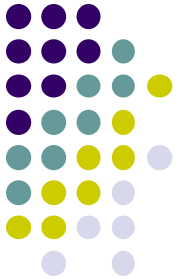


Figure 9.16 The encoder-decoder architecture. The context is a function of the hidden representations of the input, and may be used by the decoder in a variety of ways.

- **Encoder**
  - takes input sequence $x_{1:n}$ and creates a context (code) vector c
    - Context vector c conveys the essence of the input (in encoded representation).
    - This is an example of representation learning that learns new representation (often lower-dimension or other desirable form).
    - RNNs, LSTMs, Transformers can all be employed as encoders.
- **Decoder**
  - accepts the context vector c and generates a task specific output sequence $y_{1:m}$
    - Like encoders, decoders can be realized by any kind of sequence architecture such as RNNs, LSTMs, Transformers.

Why Separate Encoder and Decoder?



- **Capturing the input context**
  - The encoded representation summarizes the relevant information from the input.
- **Handling variable-length sequences**
  - The encoder transforms the entire sequence into a fixed-size representation, called “context vector” (hidden states).
- **Separating responsibilities**
  - The encoder focuses on understanding the input while the decoder focuses on generating the output.
- **Reducing error propagation**
  - Encoding the entire sequence before decoding helps in minimizing the impact of errors accumulated during the sequential decoding process.
- **Learning complex relationships between input and output**
  - Encoding creates an abstract representation of the input sequence that allows the decoder to learn complex relationships between input and output, (may enhance attention mechanism)

RNN Encoder-Decoder for Translation

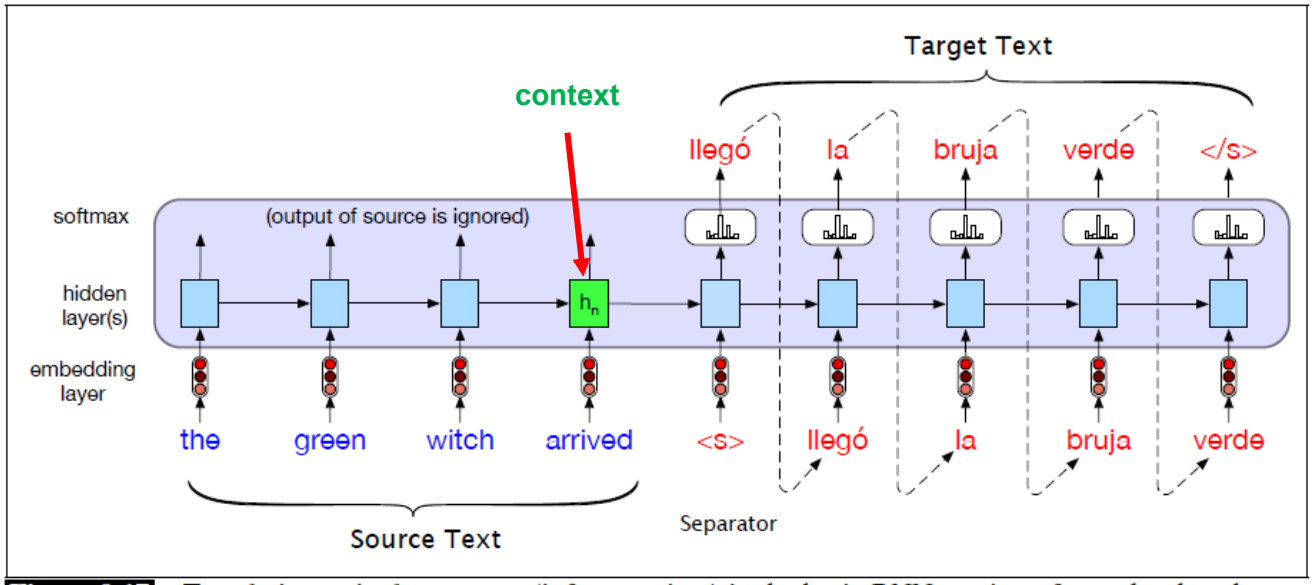


Figure 9.17 Translating a single sentence (inference time) in the basic RNN version of encoder-decoder approach to machine translation. Source and target sentences are concatenated with a separator token in between, and the decoder uses context information from the encoder's last hidden state.

- **Encoder-decoder translation at *inference time***
  - Subsequent words are conditioned on the previous hidden state.
  - Output tokens are generated autoregressively (not teacher forcing).
    - Next words are generated based on previously predicted words.
  - Output sequence is generated when the last input token is received.

RNN Encoder-Decoder for Translation

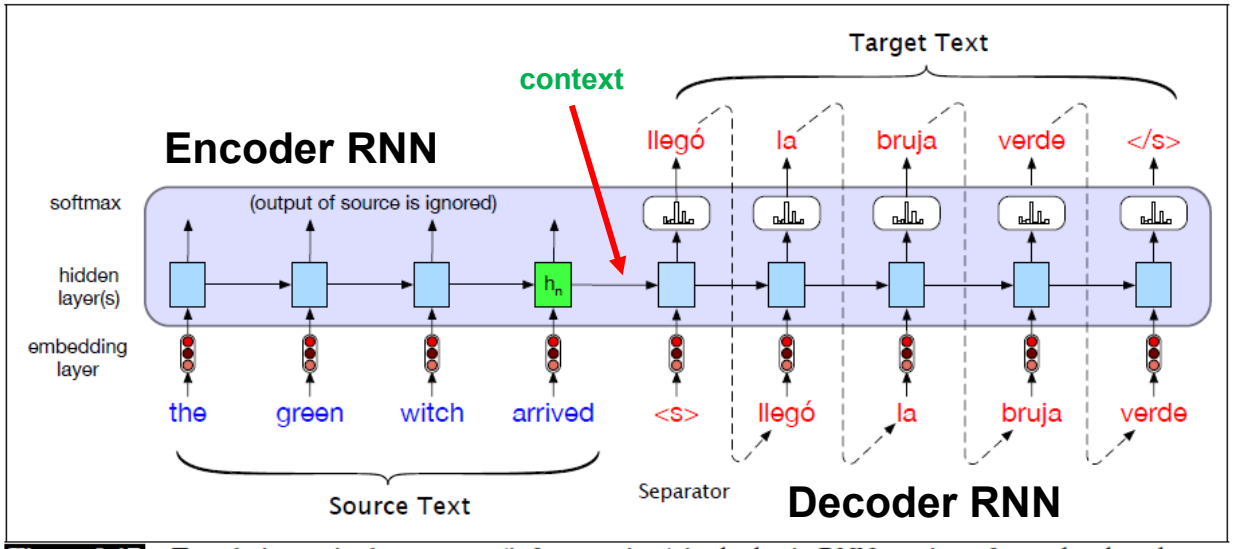


Figure 9.17 Translating a single sentence (inference time) in the basic RNN version of encoder-decoder approach to machine translation. Source and target sentences are concatenated with a separator token in between, and the decoder uses context information from the encoder's last hidden state.

- **Single network layer of encoder-decoder**
  - $h_t = g(h_{t-1}, x_t)$ (hidden states at timestep t)
  - $c = h_n^e$ (context vector in encoder that is passed to the decoder)
  - $y_t = f(h_t)$ (decoder autoregressively generates a sequence of outputs.)

Training RNN for Machine Translation

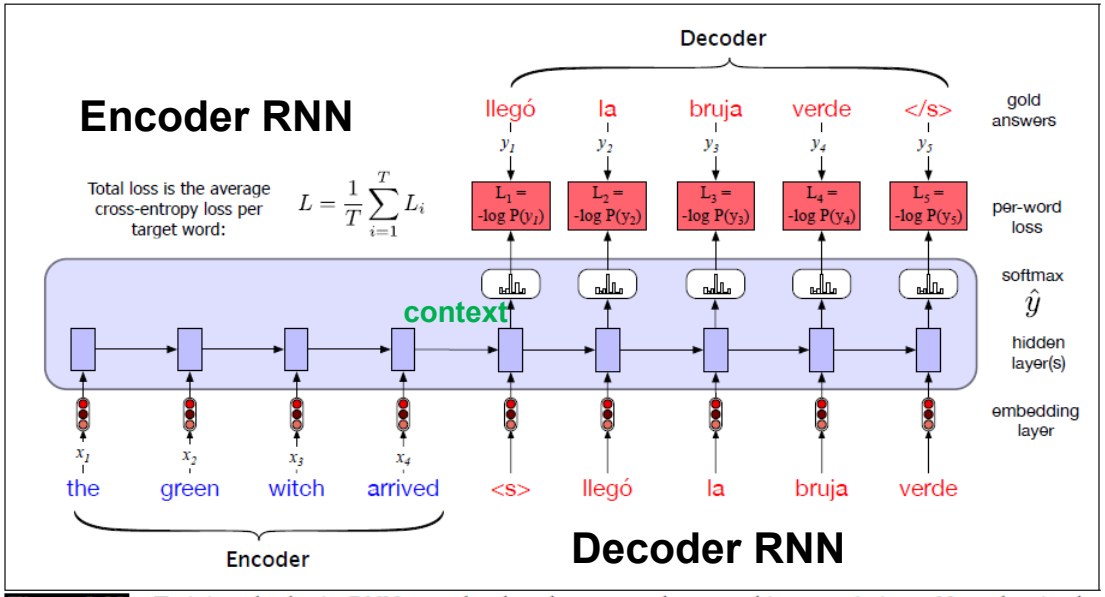


Figure 9.20 Training the basic RNN encoder-decoder approach to machine translation. Note that in the decoder we usually don't propagate the model's softmax outputs $\hat{y}_t$, but use **teacher forcing** to force each input to the correct gold value for training. We compute the softmax output distribution over $\hat{y}$ in the decoder in order to compute the loss at each token, which can then be averaged to compute a loss for the sentence.

- **Training data**
  - The training data can be drawn from standard datasets of aligned sentence pairs.
- **Training RNN**
  - The training proceeds as with any RNN-based LM (end-to-end training)

Bottleneck in Context Vector

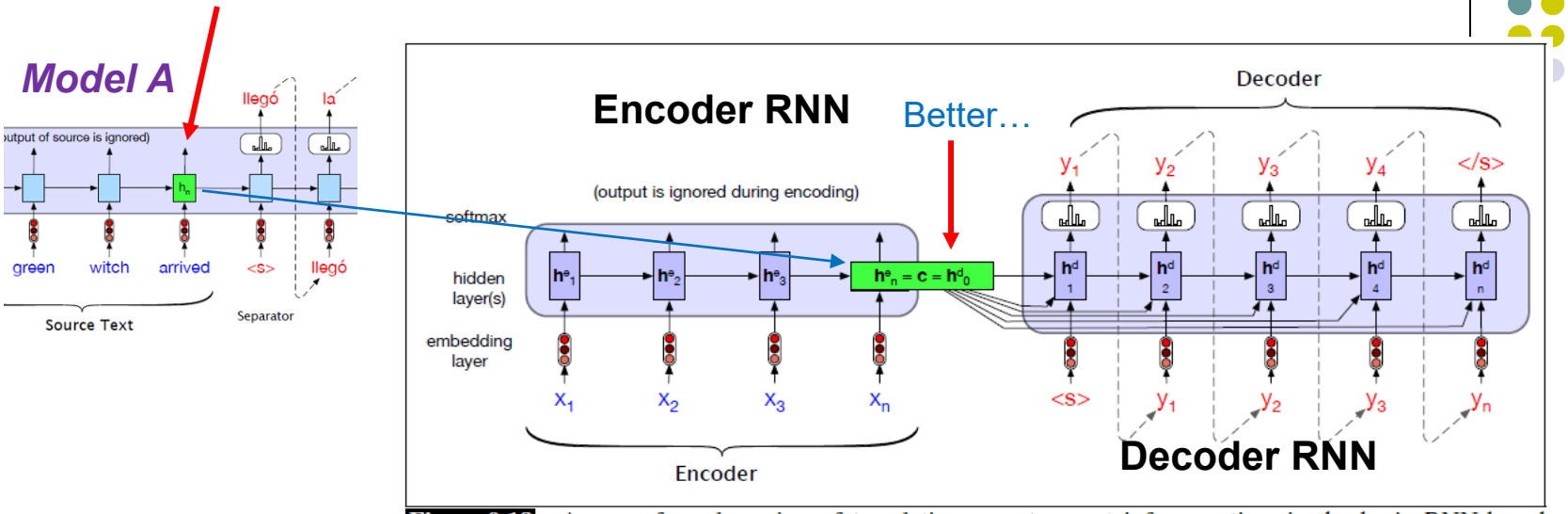
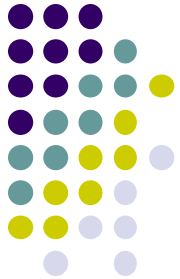


Figure 9.18 A more formal version of translating a sentence at inference time in the basic RNN-based encoder-decoder architecture. The final hidden state of the encoder RNN, h_n^e, serves as the context for the decoder in its role as h_0^d in the decoder RNN.

- Single network layer of encoder-decoder
  - $h_t = g(h_{t-1}, x_t)$ (hidden states at timestep t)
  - $c = h_n^e$ (context vector in encoder that is passed to the decoder)
  - $y_t = f(h_t)$ (decoder autoregressively generates a sequence of outputs.)
- **The *problem* of the context vector in *model A*:**
  - The hidden state of the last time step of the source text is a bottleneck.
  - The influence of the context vector c will **wane** as the output sequence is generated.

Context Vector at Each Decoder Step



- **Now the context vector $\boldsymbol{c}$ is available at each step of the decoder.**

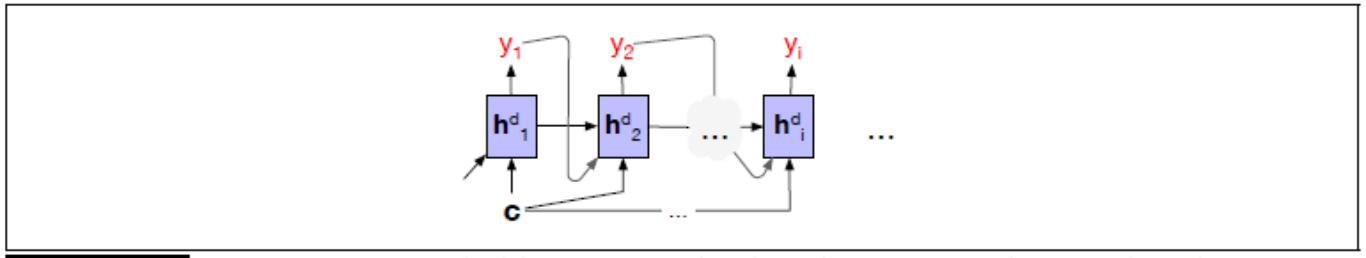
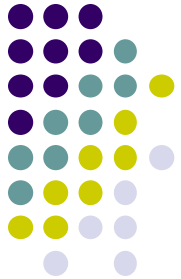


Figure 9.19 Allowing every hidden state of the decoder (not just the first decoder state) to be influenced by the context c produced by the encoder.

- $h_i^d = \boldsymbol{c}$ where $c = h_n^e$ (context from encoder)
- $h_t^d = g(\hat{y}_{t-1}, h_{t-1}^d, \boldsymbol{c})$
- $\boldsymbol{y_t} = softmax(z(h_t^d))$

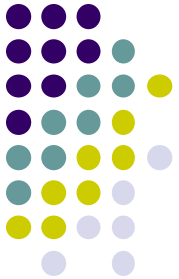
- **Compute** the most likely output at each time step:
  - $\hat{y}_t = argmax_{w \in V} P(w|x, y_1 \ldots y_{t-1})$

Pros and Cons of RNN



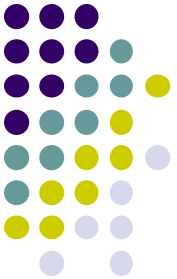
- **Advantages**
  - Ability to process variable-length input
  - Model size not increasing with size of input
  - Ability to factor in historical information in learning
  - Weights are shared across time.

- **Disadvantages**
  - Difficult to parallelize and slow computation
  - Suffered from vanishing gradient and exploding gradients
  - Difficult to capture long-distance relationships for a long sequence



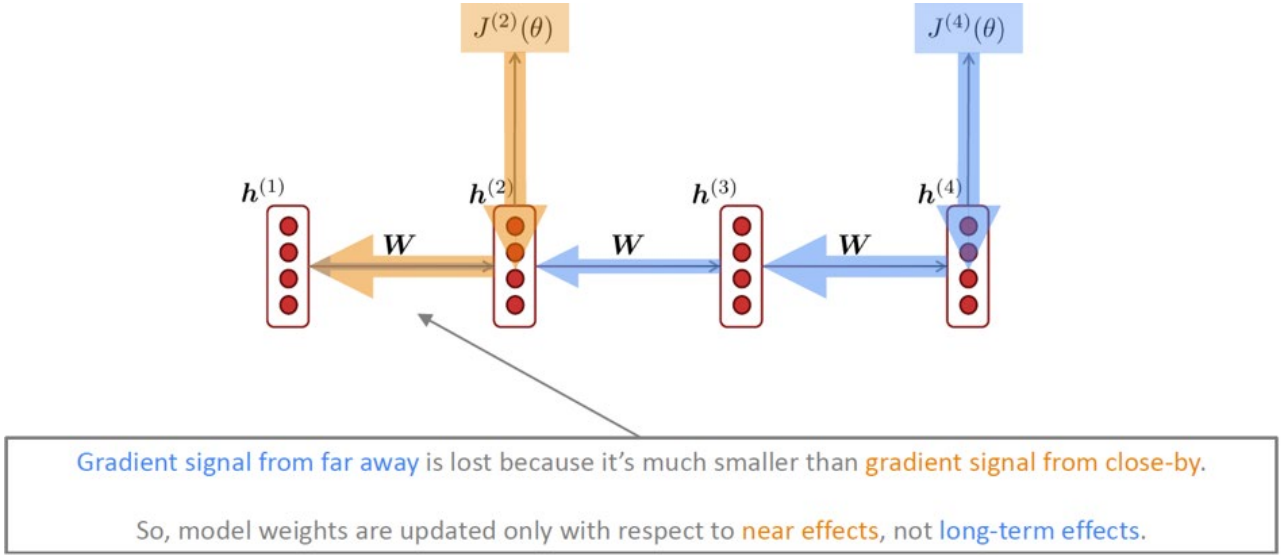
Vanishing Gradient and Exploding Gradient

Vanishing Gradient and Exploding Gradient



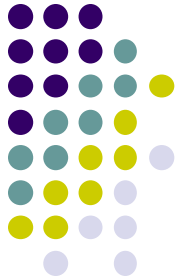
- **Vanishing gradient**
  - A product of numbers < 1.0 too many times, shrinks it to zero.
    - In the gradient by chain rule $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial H} \frac{\partial H}{\partial w}$, if each term < 1.0 and multiply many times, you get smaller and smaller numbers, getting close to zero.
  - Is vanishing/exploding gradient just a RNN problem?
    - A problem for most deep neural network architectures used for deep learning.
- **Exploding gradient**
  - A product of numbers > 1.0 or large numbers too many times, explodes to infinity, resulting in exploding gradient.
- Both problems are due to hardware limitations
- How can we resolve these problems?

Vanishing Gradient Case in RNN LMs



- **The problem of vanishing gradient in language modeling**
  - **Word prediction**: "When she tried to print her **tickets**, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her ___"
  - The RNN needs to model the dependency between "tickets" on the 7th step and the target word "tickets" at the end.
  - But if the gradient is small, the model can't learn this dependency.
  - So, the model is unable to predict similar **long-distance dependencies** at test time.

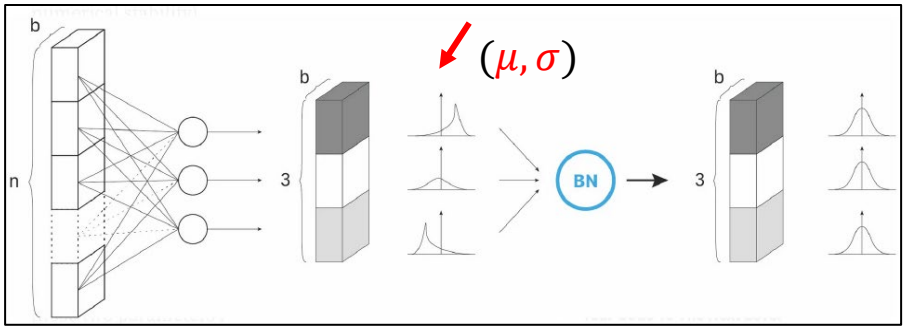
Fixing Vanishing Gradient Problem

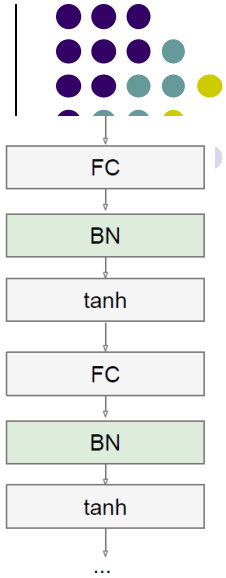


- **The root causes of vanishing gradient in RNNs**
  - During the backpropagation, the hidden layers are subject to repeated multiplications.
  - Inability to retain the distant information (long distant dependencies) and carry forward it for learning.
- **Two solution ideas**
  - Preserving past information over many time steps by having **memory**
  - Attention to key parts in the sequence
- **Some other ideas used in deep network models**
  - Simpler activation function (e.g., ReLU).
  - Batch normalization or layer normalization
  - Residual (skip) connections
    - Creating more direct and linear pass-through connections in model (ResNet)
    - Dense connections (DenseNet), highway connections (HighwayNet), etc.

Batch Normalization (BN)

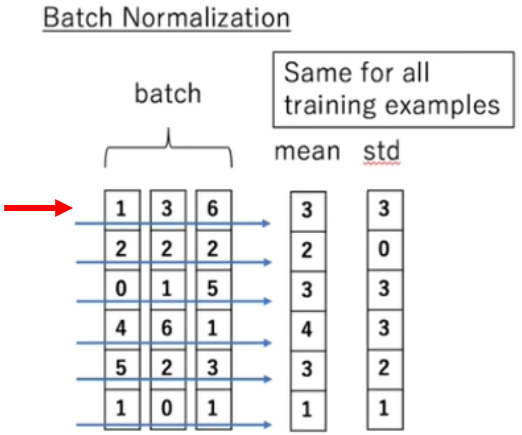
(S. Loffe and C. Szegedy, 2015)



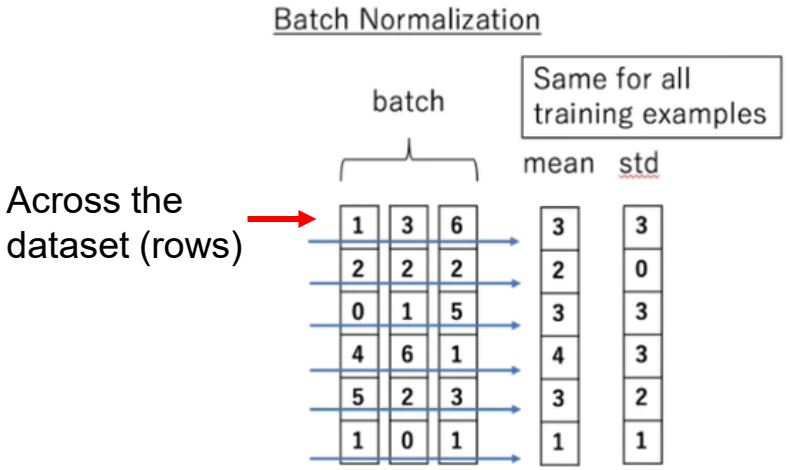


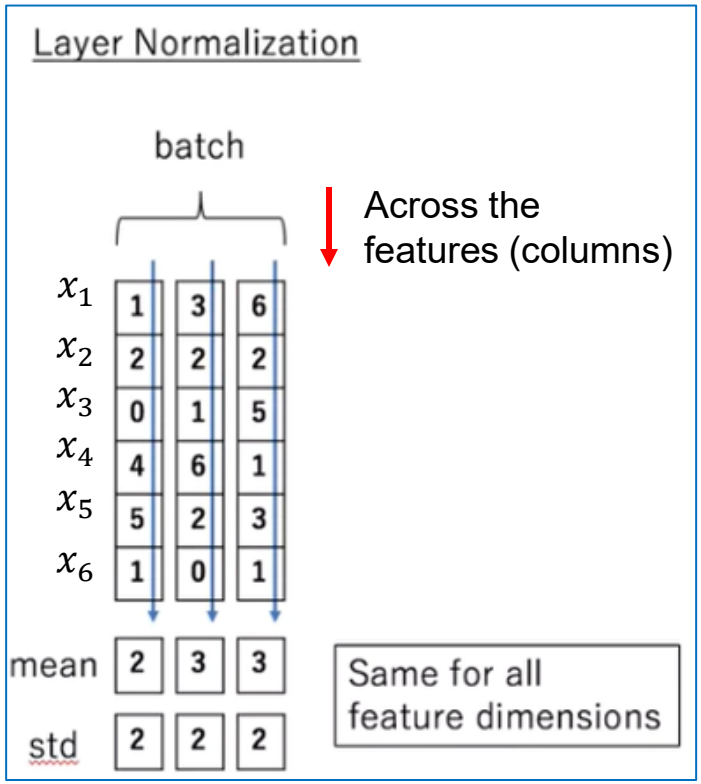
- **Basic idea**
  - Normalizing (with zero-mean) each feature of **input** within the minibatch at the beginning of each hidden layer. **Why?**
    - May help mitigate the "internal covariate shift" (the change in the distribution of the inputs to a layer)
    - Useful to deep neural networks

- **Advantages**
  - Reduced dependence on initialization
  - Improved training stability
  - Regularization effect
  - Increased learning rate



Layer Normalization

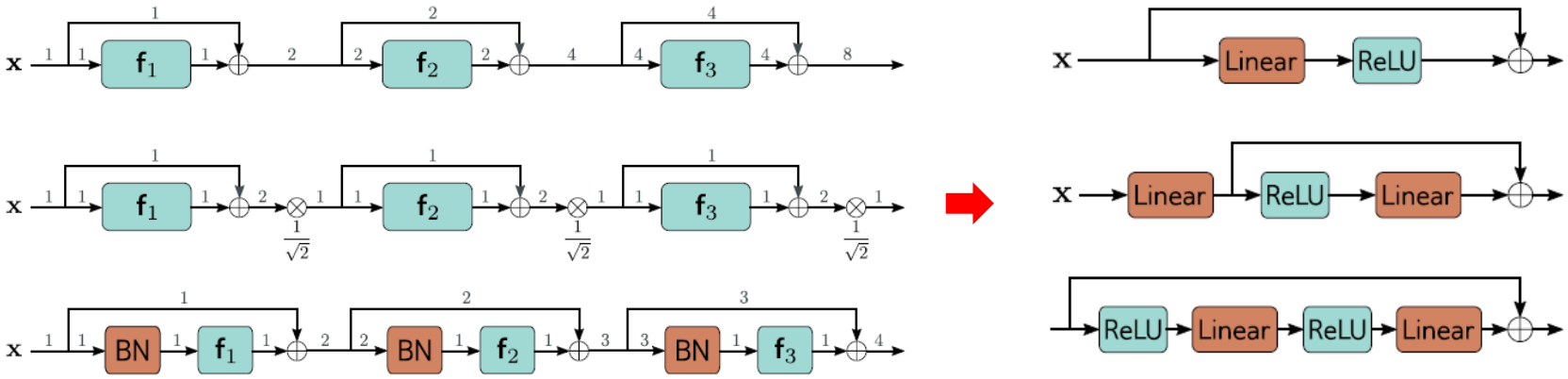




Basic idea of layer normalization
Normalize the activations across the **features** of a single sample (not across the batch).

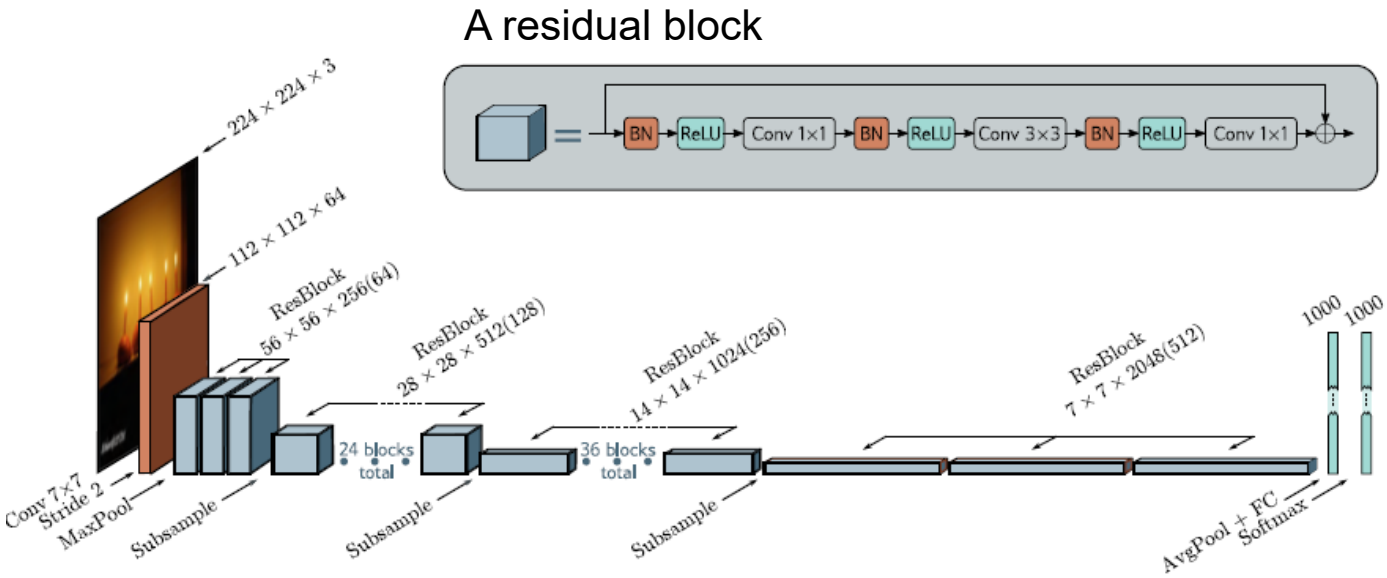
- **Batch normalization vs Layer normalization**
  - Batch normalization: normalization for each *batch* sample
  - *Layer normalization*: normalization for each feature (common in NLP)
    - Why layer normalization works better in NLP tasks?

Residual Connections and Blocks



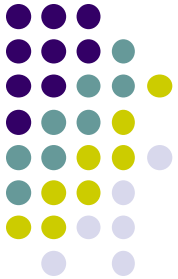
- **Residual or skip connections**
  - Branches in the computational path, whereby the input to each layer is added to the output
  - Adding residual connections roughly doubles the depth of a network
    - Studies show that BN without residual connections may cause exploding gradients.
- **Residual blocks**
  - A residual block is a building block of a residual network with residual connections
  - The stacked layers try to learn the mapping for the residual, allowing much deeper networks => perform better.

RestNet (Kaiming He et al., 2015)



- **Residual Networks (ResNets)**
  - Very deep (100s-1000s layers) network for computer vision
    - Different versions and pre-trained ResNet models available
  - The input to the block is passed through a series of convolutional layers, typically followed by batch normalization and ReLU as activation function.
- **Key innovation**
  - Use of residual blocks based on skip connection to mitigate the vanishing gradient and exploding gradient problems

Fixing Exploding Gradient Problem



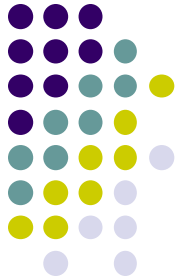
- **Gradient clipping**
  - If the norm (magnitude) of the gradient is greater than some threshold, scale it down before applying SGD update:

Algorithm 1 Pseudo-code for norm clipping

$\hat{\mathbf{g}} \leftarrow \frac{\partial \mathcal{E}}{\partial \theta}$
if $\|\hat{\mathbf{g}}\| \geq threshold$ **then**
  $\hat{\mathbf{g}} \leftarrow \frac{threshold}{\|\hat{\mathbf{g}}\|}\hat{\mathbf{g}}$
end if

  - **Intuition:** Taking a step in the same direction, but a smaller step

- In practice, exploding gradients are an easy problem to solve.



RNNs with Memory: LSTM and GRU

Long Short-Term Memory (LSTM)

(S. Hochreiter and J. Schmidhuber, 1997 but a crucial part of LSTM developed by Gers et al., 2000, and became well-known after Hinton at Google, 2013)

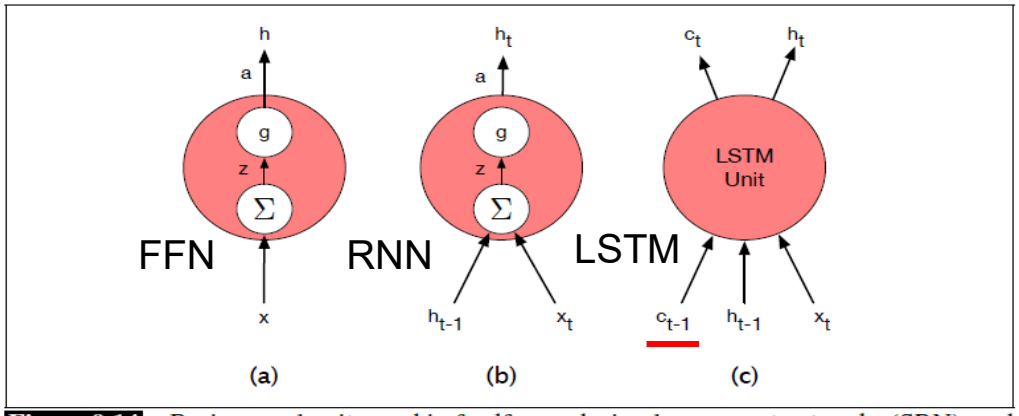
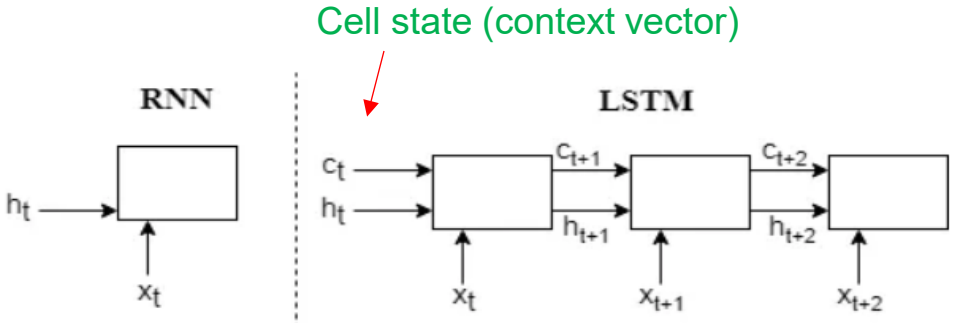
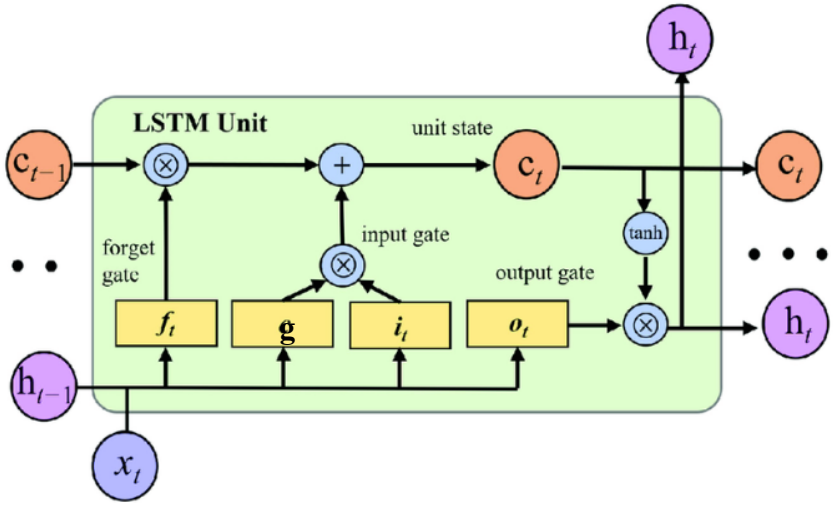


Figure 9.14 Basic neural units used in feedforward, simple recurrent networks (SRN), and long short-term memory (LSTM).

- **LSTM architecture**
  - A type of RNN designed to address the *vanishing gradient problem* and to *capture long-distance dependencies*
- **LSTM unit (cell)** has an additional input called "cell state" c that is an internal memory where context information is stored.



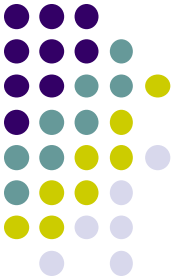
Components of LSTM Unit



- **To capture context information**
  - Three "gates" control the flow of information into and out of the units.
  - At each time step t, the cell state c_t and hidden state h_t are updated to capture the context information.

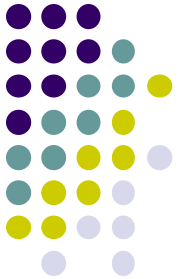
- **LSTM gates controlling information flow**
  - **Forget gate**: $f_t = \sigma(U_f h_{t-1} + W_f x_t)$ where sigmoid function σ returning a value [0,1]
  - Input gate: $i_t = \sigma(U_i h_{t-1} + W_i x_t)$
  - Output gate: $o_t = \sigma(U_o h_{t-1} + W_o x_t)$
- **LSTM states**
  - **Cell state**: $c_t = i_t \odot g_t + c_{t-1} \odot f_t$ where candidate $g_t = tahn(U_g h_{t-1} + W_g x_t)$ in [-1, 1]
    - $\odot$ is element-wise multiplication (or Hadamard product).
  - Hidden state: $h_t = o_t \odot tahn(c_t)$
  - Why tanh()? It allows zero-centered both positive and negative values.

LSTM Gates



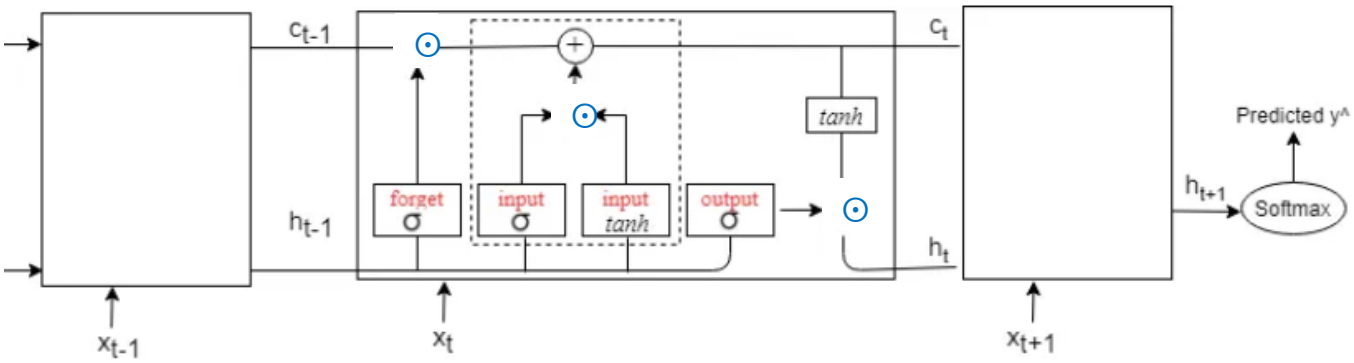
- **Forget gate**: $f_t = \sigma(U_f h_{t-1} + W_f x_t)$
  - Learns to adjust its output based on the relevance of the information contained in the current input x and previous cell state.
  - If the current input x and previous hidden state h_{t-1} contain highly relevant information to the current time step t, the forget gate is likely to produce output close to 1, otherwise 0.
  - The output of the forget gate is element-wise multiplied with the previous cell state c_{t-1} to update the cell state by $c_t = i_t \odot g_t + \boldsymbol{c_{t-1}} \odot \boldsymbol{f_t}$
- **Input gate**: $i_t = \sigma(U_i h_{t-1} + W_i x_t)$
  - Decides which values in the cell state to update with new information from the current input as used in updating the cell state $c_t = \boldsymbol{i_t} \odot g_t + \boldsymbol{c_{t-1}} \odot \boldsymbol{f_t}$.
- **Output gate**: $o_t = \sigma(U_o h_{t-1} + W_o x_t)$
  - Controls how much information from the current cell state c_t is passed on to the current hidden state h_t that is the output of the LSTM at the current time step as used in updating $h_t = \boldsymbol{o_t} \odot tahn(c_t)$.

LSTM States: Capturing the Context



- **Cell state (context vector)**: $c_t = i_t \odot g_t + c_{t-1} \odot f_t$
  - The new cell state c_t is updated by combining the information from the previous cell state c_{t-1} (retained by the forget gate f_t) and the new information from the input gate i_t and candidate cell state g_t.
  - The "candidate cell state" g_t is computed by $g_t = tahn(U_g h_{t-1} + W_g x_t)$
    - Previous information can be erased (by forget gate) or new information can be added when current input and previous state hidden state are relevant.
  - The cell state is the long-term memory of LSTM.
- **Hidden state (vector)**: $h_t = o_t \odot tahn(c_t)$
  - Determine the output of the LSTM at the current time step by element-wise multiplying the output gate $o_t = \sigma(U_o h_{t-1} + W_o x_t)$ and the current cell state c_t.
- **How is the context information captured?**
  - The context information is captured and updated through the computation of the cell state c_t and hidden state h_t at each time step.

LSTM Architecture



- **Learning sequences**
  - For a sequence of input x_t, at each time step t, compute a sequence of hidden states h_t and cell states c_t.
- **Multilayered LSTM**
  - Like simple RNNs, LSTM can be multi-layered networks with units unrolled into deep feedforward networks and trained with the usual backpropagation.
    - If LSTM unit is encapsulated as a unit, the only additional external complexity over the RNN is additional context vector as input and output.

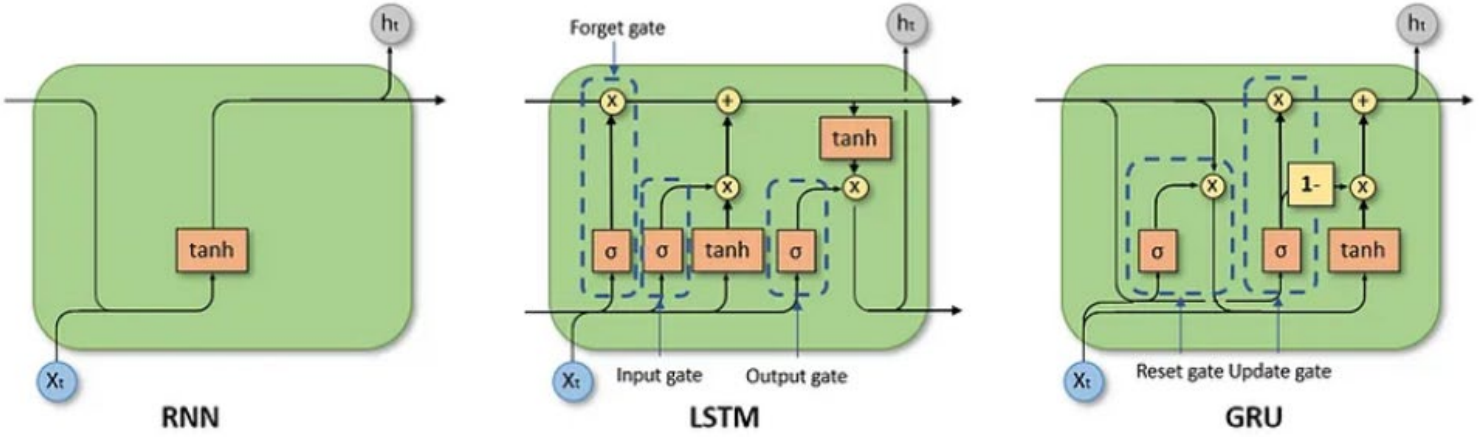
Gated Recurrent Unit (GRU)

(Kyunghyun Cho et al., 2014)



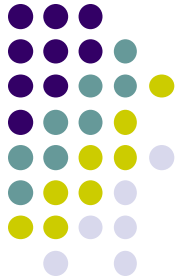
- Similar to LSTM but simpler architecture: two gates and no cell state (memory)
- **Reset gate:** $r_t = \sigma(U_r h_{t-1} + W_r x_t)$
  - Determines how much of the previous hidden state should be combined with the current input to generate the updated state.
    - A value close to 0: forget the previous state, 1: keep the previous state
- **Update gate**: $z_t = \sigma(U_z h_{t-1} + W_z x_t)$
  - Decides how much of the previous hidden state should carry over to the next state.
- **Hidden state**: $h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t)$
  - **Generates** the candidate hidden state by $\tilde{h}_t = tanh(U(r_t \odot h_{t-1}) + W x_t)$
    - $r_t \odot h_{t-1}$ indicates that the reset gate modulates the influence of the previous hidden state, potentially dropping irrelevant information from the past like forget gate in LSTM.
  - **Combines** the previous hidden state with the new candidate hidden state, with the proportion determined by the update gate.
    - If the update gate z_t is close to 0, the hidden state is more influenced by the current input and the reset-modified previous state, making it more responsible to short-term dependencies.

Comparing RNN, LSTM, and GRU Cells



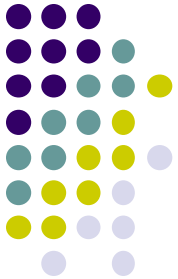
- **GRU** is similar to LSTM but simpler architecture.
  - GRU is easier and faster to train.
- **How is the context information maintained?**
  - The context information is implicitly maintained within the hidden state by combining the current input with the previous state, no separate memory cell unlike LSTM.

Limitations of LSTM and GRU



- **Vanishing/exploding gradients**
  - Although both architectures are designed to mitigate the problem of RNN, they still suffer from vanishing/exploding gradients, unable to capture long-distance dependencies.
  - What is the root cause of this problem?
    - Input sequence is processed sequentially, one element at a time in a sequence.
- **Difficult to parallelize the processing**
  - Sequential nature of processing input, e.g., each element depending on the outputs and the hidden states of the previous step, making it difficult to parallelize the processing
- **Fixed context window**
  - Ability to remember information from previous timesteps is limited by its hidden and cell state sizes, unable to capture long-distance dependencies

LSTM: Real-world Applications



- **LSTM state-of-the-art results** (2013 – 2015)
  - Successful for pattern generation, sequence analysis, sentiment analysis, handwriting recognition, speech recognition, machine translation, parsing, and image captioning, as well as language models.
  - LSTMs became the dominant approach for most NLP tasks.
- **Drawbacks**
  - Complex structure – more computation and longer training time
  - The problem of capturing long-distance dependencies due to vanishing/exploding gradients still exist.
- **Transformers** have become **dominant** for all tasks (2019 –)
  - In WMT (Workshop on Machine Translation) conference + competition:
    - In WMT 2014, there were 0 neural machine translation systems!
    - In WMT 2016, the summary report contains “RNN” 44 times that won.
    - In WMT 2019, “RNN” 7 times, “Transformer” 105 times.